Praxis Thesis Corpus

Every theorem and proof, one validated graph

Sean Chatman

Contents

Chapter 1

Foundations

Axiom 1.0.1 (ax:obs)**.** *There is a set $\mathcal{O}$, the observation space, whose elements are arbitrary finite records; $\mathcal{O}$ carries no decidable semantics.*

Construction 1.0.2 (con:denial)**.** *$D = (\{0,1\}^n, \vee, \mathbf{0})$ is the commutative idempotent monoid of denial words; the total denial of an observation is $d(o) = \bigvee_i d_i(o)$, and $\alpha(o) \neq \bot \iff d(o) = \mathbf{0}$.*

Corollary 1.0.3 (cor:antibody)**.** *Under* BRCE *the overclaim set (actuations lacking a valid receipt) is empty as an invariant: the admissible magnitude of any claim is bounded by what the mechanism can receipt.*

Corollary 1.0.4 (cor:noadmit)**.** *There is no algorithm that admits an observation by deciding a non-trivial semantic property of what it denotes; any total admission procedure decides a syntactic, decidable surrogate instead.*

Corollary 1.0.5 (cor:subcat)**.** *The sub-category of admissible morphisms a program may actually perform is precisely* **Life***; illegal-transition-is-a-type-error is a static fact about inhabited hom-sets, not a runtime check.*

Definition 1.0.6 (def:adm)**.** Fix a decidable sub-collection $\mathcal{O}^* \subseteq \mathcal{O}$ and a finite battery of total, computable obligations $g_1, \ldots, g_m$; the admission map α is the partial retraction sending o to $\rho(o) \in \mathcal{O}^*$ if all $g_i(o) = 1$, else to $\bot$.

Definition 1.0.7 (def:brce)**.** A system satisfies the Bounded Receipted Chatman Equation if for every actuated artifact it maintains the admission gate (B1), bounded manufacture (B2), receipt totality (B3), and conformance (B4).

Definition 1.0.8 (def:fit)**.** Conformance fitness $\varphi \in [0,1]$ is one minus the fraction of tokens a replay was forced to consume on unenabled lifecycle transitions; $\varphi = 1$ iff the replay never attempted a disabled firing.

Definition 1.0.9 (def:five)**.** The Chatman equation relates five objects: the observation space $\mathcal{O}$, the admitted space $\mathcal{O}^*$, the manufacturing morphism μ, the artifact/action space $\mathcal{A}$, and the receipt space $\mathcal{R}$.

Definition 1.0.10 (def:lifecat)**.** **Life** is the free category on the linear quiver $\mathsf{Raw} \xrightarrow{j} \mathsf{Validated} \xrightarrow{a} \mathsf{Admitted} \xrightarrow{r} \mathsf{Receipted}$, with four objects and three generating arrows judge, admit, receipt.

Definition 1.0.11 (def:mu)**.** A manufacturing morphism is a deterministic computable map $\mu : \mathcal{O}^* \to \mathcal{A}$ satisfying determinism/reproducibility (M1) and boundedness (M2); μ is undefined off $\mathcal{O}^*$ and $\mu(\bot) = \bot$.

Definition 1.0.12 (def:receipt)**.** Fix a collision-resistant hash H; the frame is $\mathsf{fr} = \langle \theta, \mathsf{H}(b) \rangle$ and the chained receipt is $h_+ = \mathsf{H}(h_- \| \mathsf{fr})$; the receipt is the $\leq \kappa$-chunk tuple (verdict, h_+, φ, reason).

Definition 1.0.13 (def:seqtoken)**.** The lifecycle process is modeled as a safe Petri net (3-Node SEQ POWL token game) over the marking $M = (\texttt{TOK\_START}, \texttt{TOK\_JUDGED}, \texttt{TOK\_ADMITTED}, \texttt{TOK\_DONE})$, with fitness $\varphi \in [0, 1]$ in Q16.16.

Definition 1.0.14 (def:taxonomy)**.** A refusal taxonomy is a pair (S, cat) where S is a finite set of concrete refusal scenarios and $\mathrm{cat} : S \to C$ is a total map onto the eight-element category set C.

Lemma 1.0.15 (lem:commit)**.** *Under collision resistance of* H*, the chain value* h_+ *is a binding commitment to* (h_-, fr)*; by induction* h_+ *binds the entire causal prefix.*

Proposition 1.0.16 (prop:hom)**.** *For stages* X, Y *of* **Life***, the hom-set* $\mathbf{Life}(X, Y)$ *has exactly one element if* Y *is reachable from* X *along the quiver, and is empty otherwise.*

Proposition 1.0.17 (prop:mudet)**.** *Under (M1),* $x \mapsto \mu(x)$ *is single-valued, so* $o \mapsto \mu(\alpha(o))$ *is a partial function on* $\mathcal{O}$*: two runs on the same admitted input agree bit-for-bit.*

Proposition 1.0.18 (prop:retract)**.** α *restricted to* $\mathcal{O}^*$ *is the identity, and* $\alpha \circ \alpha = \alpha$ *as partial maps; hence* $\mathcal{O}^*$ *is a retract of* $\mathrm{dom}(\alpha)$*.*

Proposition 1.0.19 (prop:semilattice)**.** $(\{0,1\}^n, \vee, \mathbf{0})$ *is a bounded join-semilattice; consequently adding obligations can only enlarge the denial, never shrink it.*

Proposition 1.0.20 (prop:total)**.** *The category map* $\mathrm{cat} : S \to C$ *is total (enforced by the host type system's exhaustive match), and the lane map's single-lane inverse is a section of the lane map over the seven named lanes.*

Theorem 1.0.21 (thm:conservation)**.** *Under* BRCE*, the map* $a \mapsto h_+(a)$ *from actuated artifacts to receipt-chain positions is well defined and injective up to hash collision, and every actuated artifact is the image under* μ *of exactly one admitted observation.*

Theorem 1.0.22 (thm:rice)**.** *Rice's theorem, specialized to observations: for any non-trivial semantic property* P *of the function an observation denotes, the set* $\{o \in \mathcal{O} : P(o)\}$ *is undecidable.*

Theorem 1.0.23 (thm:typestate)**.** *Interpreting each lifecycle stage as a LawObject type family, the exposed stage-changing operations denote exactly the generating arrows of* **Life***; every well-typed stage-changing program is a composite of* j, a, r*, illegal transitions are uninhabited hom-sets, and no law object can be receipted twice.*

Proof of 3.0.5. (B3) requires a receipt per actuation; an actuation without one violates BRCE and is, by (B1)'s gate, never emitted, so the set of receiptless actuations is empty. *(by def:brce)*
□

Proof of 1.0.4. A total admission deciding a non-trivial semantic property would be a decider for an undecidable set; a total terminating procedure exists only for decidable predicates, the largest such structure being a decidable sub-collection onto which admission maps. *(by thm:rice)*
□

Proof of 1.0.5. Combining that the inhabited stage-changing types are exactly the morphisms of **Life** and the uninhabited ones are exactly the empty hom-sets, the admissible sub-category is **Life** itself, verified statically. *(by thm:typestate)*
□

Proof of 1.0.15. If $\mathsf{H}(h_- \| \mathsf{fr}) = \mathsf{H}(h'_- \| \mathsf{fr}')$ for distinct arguments, the two arguments are a collision of H, occurring with negligible probability under collision resistance. *(by def:receipt)*
Since h_- is itself such a commitment to the previous frame, induction on chain length shows h_+ commits to every frame in the prefix. *(by def:receipt)*
□

Proof of 1.0.16. In the free category on a quiver, morphisms are directed paths; the quiver here is a simple directed path with no branches or cycles, so between any two vertices there is at most one directed path, giving singleton or empty hom-sets. *(by def:lifecat)*
□

Proof of 1.0.17. (M1) is exactly single-valuedness; composing the partial function α with the function μ yields a partial function, and bit-identity of outputs on equal inputs is the contrapositive of disagreement implying differing input. *(by def:mu)*
□

Proof of 1.0.18. For $x \in \mathcal{O}^*$ every $g_i(x) = 1$ by construction, so $\alpha(x) = \rho(x) = x$, i.e. $\alpha|^*_{\mathcal{O}} = \mathrm{id}^*_{\mathcal{O}}$. *(by def:adm)*
For general o with $\alpha(o) \neq \bot$, $\alpha(o) \in \mathcal{O}^*$, whence $\alpha(\alpha(o)) = \alpha(o)$; idempotence follows on all of $\mathrm{dom}(\alpha)$. *(by def:adm)*
□

Proof of 6.0.17. Idempotence, commutativity, and associativity of $\vee$ hold coordinatewise, so a product of semilattices is a semilattice with $\mathbf{0}$ the identity. *(by con:denial)*
If $G' \supseteq G$ then $d_{G'}(o) \succeq d_G(o)$ coordinatewise, and since $\mathbf{0}$ is the unique minimum, $\{o : d_{G'}(o) = \mathbf{0}\} \subseteq \{o : d_G(o) = \mathbf{0}\}$: adding obligations only shrinks the admitted set. *(by con:denial)*
□

Proof of 1.0.20. Totality of a wildcard-free exhaustive match over a closed enum is exactly the compiler's exhaustiveness guarantee, so cat denotes a total function $S \to C$. *(by def:taxonomy)*
The section identity is the round-trip property: for each of the seven single-lane constants ℓ, $\mathrm{scn}(\ell)$ is defined and $\mathrm{lane}(\mathrm{scn}(\ell)) = \ell$; a composed multi-lane word has no single scenario, per the denial monoid's join structure. *(by con:denial)*
□

Proof of 6.0.18. By (B1) actuation implies $a = \mu(x)$ with $x = \alpha(o) \neq \bot$, so $\mathrm{im}\,\mu$ is the whole actuated set and its complement is never actuated. *(by def:brce)*
By (B2)/(M1), $x \mapsto \mu(x)$ is single-valued, so each actuated a has a determined admitted preimage x. *(by prop:mudet)*
By (B3) the actuation appends exactly one chain position, so $a \mapsto h_+(a)$ is a well-defined total map; injectivity up to collision follows because two distinct actuations sharing a chain value would collide under H. *(by lem:commit)*
Order preservation is the recursive dependence of h_+ on h_- in the chain equation. *(by def:receipt)*
□

Proof of 6.0.22. This is Rice's theorem instantiated at $\mathcal{O}$: a decider for $\{o : P(o)\}$ would decide the halting problem via the reduction $o_{M,x}$, a contradiction. *(by thm:rice)*
□

Proof of 1.0.23. The only public constructor produces a Raw object and the only public stage-changing operations are judge, admit, receipt, whose types are j, a, r; the stage-rebuilding helper is crate-private, so every externally well-typed stage-changing program is a composite of j, a, r, i.e. a morphism of **Life**. *(by prop:hom)*
If $\mathbf{Life}(X, Y) = \varnothing$ there is no path $X \to Y$ in the quiver, so no composite of j, a, r has the required type; concretely receipt is impl'd only on Admitted and admit takes Validated by value, so the mismatched call is a type error. *(by prop:hom)*
receipt takes self by value, consuming the Admitted object, and the returned Receipted type has no further stage-changing method, so a second receipting is both unconstructible and unavailable. *(by def:lifecat)*

□

Chapter 2

Admission Algebra

Axiom 2.0.1 (ax:refusal)**.** *There is a distinguished refusal value $\bot$ and a space of reasons $\mathcal{R}_f$; a refusal is not the absence of an output but the pair $(\bot, r)$ with $r \in \mathcal{R}_f$ a machine-checkable reason, so admission is total as a map into $\mathcal{O}^* \cup (\{\bot\} \times \mathcal{R}_f)$.*

Corollary 2.0.2 (cor:allstages)**.** $\Phi_o(w) = \mathsf{ADMITTED} \iff \varphi_o(s_i) = \mathsf{ADMITTED}$ *for every stage s_i in w; a single refusing stage refuses the whole pipeline, and the aggregate word records every lane that fired anywhere along w.*

Corollary 2.0.3 (cor:assembled)**.** *Composing the total maps of prop:obtotal and prop:section with the total classification of thm:total: every unmet obligation (via* $\mathrm{cat} \circ \mathrm{scn}$*) and every non-*ADMITTED *single denial lane (via* $\mathrm{cat} \circ \mathrm{scn}_\ell$ *on* $L \setminus \{\mathsf{ADMITTED}\}$*) maps to exactly one of the seven inhabited categories; no obligation failure and no fired single lane is left unclassified.*

Corollary 2.0.4 (cor:galois)**.** *The assignment $G \mapsto \mathcal{O}^*_G$ is an order-reversing map from $(2^{\mathcal{G}}, \subseteq)$ to $(2^{\mathcal{O}}, \subseteq)$: $G \subseteq G' \Rightarrow \mathcal{O}^*_{G'} \subseteq \mathcal{O}^*_G$; the empty obligation set admits everything ($\mathcal{O}^*_\varnothing = \mathcal{O}$), and enlarging obligations monotonically tightens the gate.*

Corollary 2.0.5 (cor:mrrindep)**.** *thm:mrrsep holds exactly when no obligation in G_{prop} couples distinct accounts; if such coupling exists, the proposer's obligation battery must be extended with a BlockingConstraint on the shared resource, which by thm:mono only tightens the admitted set and preserves the denial algebra.*

Definition 2.0.6 (def:decoder)**.** Let $L = \{\mathsf{ADMITTED}, c_1, \ldots, c_7\} \subseteq \mathbb{D}$ be the clean word together with the seven named single-lane constants; the lane decoder $\mathrm{scn}_\ell : \mathbb{D} \rightharpoonup \mathcal{S}$ is scenario_for_denial_lane: it returns None on ADMITTED, returns the matching denial-lane scenario on each c_i, and returns None on every word not in L.

Definition 2.0.7 (def:denialabstract)**.** Fix $n \in \mathbb{N}$ lanes; the abstract denial word space is $\mathbb{D}_n = \{0,1\}^n$ with componentwise disjunction $\vee$ and the all-zero word $\mathbf{0}$; lane i is set in d when $d_i = 1$, $\mathrm{supp}\, d = \{i : d_i = 1\}$, and $d \preceq d'$ means $\forall i\, (d_i \le d'_i)$.

Definition 2.0.8 (def:denialcode)**.** In praxis the denial word is DenialPolarity, a repr(transparent) newtype over u64 carved into eight byte-lanes; the clean word is ADMITTED = DenialPolarity(0); seven named nonzero constants each occupy one distinct byte lane; the product is $\mathrm{compose}(a, b) = \mathrm{DenialPolarity}(a.0 \mid b.0)$, bitwise OR; the admission predicate is $\mathrm{is\_admitted}(d) \iff d.0 = 0$.

Definition 2.0.9 (def:fired)**.** $\pi_f : \mathbb{D} \to \{0,1\}^8$ sends a word d to the bit vector whose bit j equals the nonzero-indicator of byte lane j of d: $\pi_f(d)_j = \mathbf{1}[(d.0 \gg 8j)\ \&\ \mathrm{0xFF} \neq 0]$, computed branchlessly via $\mathbf{1}[x \neq 0] = (x \mid (-x)) \gg 63$ on each lane.

Definition 2.0.10 (def:logicadm)**.** prolog8 admits a query, fact block, or rule only if it lies in a bounded, decidable Horn fragment: arity ≤ 8, rule body ≤ 8 atoms, ≤ 8 variables, proof fan-in ≤ 8; no cut, no dynamic mutation, stratified negation, bounded recursion, no runtime text parsing, no side effects, interned terms; violations are enumerated by RejectionCode.

Definition 2.0.11 (def:mrr)**.** Let A be a set of client accounts; for each $a \in A$ let lawful_targets(a) be the evidence-gated target stages and realized(a, s) the revenue function under stage s; the Maximum Reachable Revenue is $\mathrm{MRR} = \max_{\text{plan}} \sum_{a \in A} \text{realized}(a)$.

Definition 2.0.12 (def:ob)**.** An obligation is a first-class, hashable value the payload must satisfy before admission; the Obligation enum has exactly three kinds; each obligation g induces a lane map $\delta_g : \mathcal{O} \to \mathbb{D}$ returning ADMITTED if g is satisfied by o, else $\ell(g)$; for a set G the total denial is $d_G(o) = \bigvee_{i=1}^{m} \delta_{g_i}(o)$.

Definition 2.0.13 (def:obsauth)**.** Let $\mathcal{O}$ be the raw observation space; an observation $o \in \mathcal{O}$ is authoritative ($o \in \mathcal{O}^*$) if it was produced by an admitted actuation with a chained receipt; all other observations are untrusted; the proposer's admission map α_{prop} retracts $\mathcal{O}$ onto $\mathcal{O}^*_{\text{prop}}$ by treating every inbound observation as untrusted until its receipt chain satisfies the obligation battery G_{prop}.

Definition 2.0.14 (def:pipeline)**.** Let Stage be the set of admission stages; fix o and let $\varphi_o : \text{Stage} \to \mathbb{D}$ send a stage to the denial word it emits on o; a pipeline is a finite sequence $w = s_1 \cdots s_k \in \text{Stage}^*$, an element of the free monoid on Stage; its aggregate denial is $\Phi_o(w) = \bigvee_{i=1}^{k} \varphi_o(s_i)$, $\Phi_o(\varepsilon) =$ ADMITTED.

Definition 2.0.15 (def:queryresult)**.** Kernel::query returns QueryResult, one of Answered(Vec¡Decision¿), Denied(Box¡Decision¿), or Invalid(RejectionCode); a Decision carries a proof (a Vec¡ProofNode¿ DAG, positive or negative) and a receipt sufficient for deterministic replay.

Definition 2.0.16 (def:tax)**.** The category set $\mathcal{C}$ is the eight-element enum Identity/Capacity/Topology/Temporal/Li the scenario set $\mathcal{S}$ is the thirteen-variant RefusalScenario enum partitioned into three obligation-driven, seven denial-lane, and three logic/andon variants; the category map cat : $\mathcal{S} \to \mathcal{C}$ is RefusalScenario::category.

Proposition 2.0.17 (prop:bottom)**.** $\alpha_G(o) \neq \bot \iff d_G(o) = \text{ADMITTED} \iff$ *every* $g_i \in G$ *is satisfied by* o*.*

Proposition 2.0.18 (prop:boundary)**.** *Under def:obsauth, praxis-proposer enforces* $\operatorname{im} \mu_{\text{prop}} \subseteq \mathcal{O}^*_{\text{prop}}$*: no proposal is manufactured from an unadmitted observation; the generic Domain¡D¿ proposer evaluates its Obligation battery on every inbound record before calling Admit::admit, and a record failing any obligation is refused and never reaches manufacturing.*

Proposition 2.0.19 (prop:boundedcert)**.** *Every proof node emitted by the kernel has fan-in* ≤ 8 *and every rule body has* ≤ 8 *atoms, so a proof is a bounded-degree DAG; its rolling proof_root is a single fixed-width hash field of the receipt, and a bounded-context verifier checks a query outcome in* $O(\kappa)$ *by recomputing proof_root.*

Proposition 2.0.20 (prop:code_is_sl)**.** $\mathbb{D} = (\{\mathit{DenialPolarity}\}, \mathit{compose}, \text{ADMITTED})$ *is a bounded commutative idempotent monoid, is_admitted is exactly the predicate 'is the bottom element,' and the submonoid* $\langle L \rangle$ *generated by the seven named nonzero constants together with* ADMITTED *is isomorphic to* $\mathbb{D}_7$ *via 'which named lanes are nonzero.'*

Proposition 2.0.21 (prop:embed)**.** *The refusing branches of thm:trichotomy embed into* $(\mathbb{D}, \mathit{compose}, \mathsf{ADMITTED})$ *via the taxonomy: run_kernel_query maps Denied to scenario KernelDenied (category Authorization) and Invalid to KernelInvalid (category Identity); by denial_lane both compose into lane CONFORMANCE_GATE_FAILED, so a logical refusal is subject to monotonicity, classification, pipeline composition, and fired-mask projection exactly as any obligation refusal.*

Proposition 2.0.22 (prop:fleet)**.** *Let a fleet of N manufactures produce aggregate words $\Phi^{(1)}, \ldots, \Phi^{(N)}$, packed as $b^{(r)} = \pi_f(\Phi^{(r)}) \in \{0,1\}^8$; then $\pi_f(\Phi_o(w)) = \bigvee_i \pi_f(\varphi_o(s_i))$, and a fleet admission sweep is one linear pass computing $\bigvee_r b^{(r)}$, returning $\mathbf{0}$ iff every agent is admitted.*

Proposition 2.0.23 (prop:monoid)**.** $(\mathbb{D}_n, \vee, \mathbf{0})$ *is a commutative monoid in which every element is idempotent; it is the join-semilattice of the Boolean lattice $(2^{[n]}, \subseteq)$ under $d \mapsto \operatorname{supp} d$, with $\mathbf{0}$ least and $\mathbf{1}$ greatest, so $d \vee d'$ is the least upper bound and $\preceq$ is exactly $d \preceq d' \iff d \vee d' = d'$.*

Proposition 2.0.24 (prop:obtotal)**.** *The map* $\mathrm{scn} : \mathit{Obligation} \to \mathcal{S}$ *implemented by From¡&Obligation¿ for RefusalScenario is a total function realized by a wildcard-free match over the three Obligation kinds; because the match has no wildcard arm, totality is a static property of the program, not a runtime hope.*

Proposition 2.0.25 (prop:partial)**.** scn_ℓ *is a partial map, definite (as Some) only on $L \backslash \{\mathsf{ADMITTED}\}$; it is not realizable as a wildcard-free exhaustive match because its domain $\mathbb{D}$ is an open value space of cardinality 2^{64}, not a closed enum, so exhaustiveness over the seven single lanes is a runtime test obligation, not a static type obligation.*

Proposition 2.0.26 (prop:section)**.** *The lane encoder* $\mathrm{lane} : \mathcal{S} \to \mathbb{D}$ *implemented by denial_lane is total via a wildcard-free match over all thirteen scenarios; restricted to the seven denial-lane scenarios $\mathcal{S}_\ell$, it and* scn_ℓ *form a section-retraction pair, so the seven single lanes and the seven denial-lane scenarios are in bijection; the remaining six scenarios map onto closest-fit lanes and are not in that bijection's range.*

Theorem 2.0.27 (thm:firedhom)**.** *π_f is a homomorphism of bounded commutative idempotent monoids: $\pi_f(\mathsf{ADMITTED}) = \mathbf{0}$ and $\pi_f(\mathit{compose}(a,b)) = \pi_f(a) \vee \pi_f(b)$ for all $a, b \in \mathbb{D}$; restricted to $\langle L \rangle$ it is a monoid isomorphism onto the seven-bit image $\{0\} \times \{0,1\}^7$.*

Theorem 2.0.28 (thm:freehom)**.** $\Phi_o : (\mathit{Stage}^*, \cdot, \varepsilon) \to (\mathbb{D}, \mathit{compose}, \mathsf{ADMITTED})$ *is the unique monoid homomorphism extending φ_o along $\mathit{Stage} \hookrightarrow \mathit{Stage}^*$; because the target is commutative and idempotent, $\Phi_o(w)$ depends only on the set of distinct stage-denials occurring in w, invariant under reordering and repetition.*

Theorem 2.0.29 (thm:mono)**.** *Let $G \subseteq G'$ be obligation sets; then $d_G(o) \preceq d_{G'}(o)$ for every observation o, hence $\mathcal{O}^*_{G'} \subseteq \mathcal{O}^*_G$: adding obligations can only enlarge denial and shrink admission, never admit an observation a smaller obligation set refused.*

Theorem 2.0.30 (thm:mrrsep)**.** *If the accounts in A are independent, then* $\mathrm{MRR} = \sum_{a \in A} \max_{s \in \mathrm{lawful\_targets}(a)} \mathrm{realiz}$ *reducing the joint plan search from exponential to linear in* $|\mathrm{lawful\_targets}(a)|$.

Theorem 2.0.31 (thm:total)**.** *The category map* $\mathrm{cat} : \mathcal{S} \to \mathcal{C}$ *is total (every one of the thirteen scenarios has exactly one category), mechanized (compiler-certified via a wildcard-free match), and its image is $\mathcal{C} \setminus \{\mathit{Reserved}\}$: exactly seven buckets are inhabited, and Reserved has empty preimage.*

Theorem 2.0.32 (thm:trichotomy)**.** *For a query q evaluated against an admitted kernel, exactly one of Answered (a positive proof DAG with fan-in ≤ 8 plus receipt), Denied (a negative proof plus receipt), or Invalid (a RejectionCode, no proof) holds.*

Proof of 2.0.2. A join in a join-semilattice equals the bottom iff every joinand is the bottom; apply to $\Phi_o(w) = \bigvee_i \varphi_o(s_i)$; the support of the aggregate is $\operatorname{supp} \Phi_o(w) = \bigcup_i \operatorname{supp} \varphi_o(s_i)$, the union of the per-stage fired lanes. *(by prop:monoid)*

□

Proof of 2.0.3. Immediate by composition of the three total (respectively Some-total) maps, each single-valued into its codomain, landing in $\operatorname{im} \mathrm{cat} = \mathcal{C} \setminus \{\text{Reserved}\}$. *(by thm:total)*

□

Proof of 2.0.4. Order-reversal is thm:mono; with $G = \varnothing$ the join is empty, so $d_\varnothing(o) = \mathsf{ADMITTED}$ for all o and $\mathcal{O}^*_\varnothing = \mathcal{O}$. *(by thm:mono)*

□

Proof of 2.0.5. Immediate from thm:mono: adding an obligation enlarges denial and shrinks admission; with the extended battery the coupled accounts are refused by the coupling constraint and the remaining independent ones decompose additively as before. *(by thm:mono)*

□

Proof of 2.0.17. $d_G(o) = \bigvee_i \delta_{g_i}(o) = \mathsf{ADMITTED}$ iff every summand is ADMITTED (a join of elements of a join-semilattice equals the bottom iff each element is the bottom), i.e. iff no obligation is unmet; admission returns non-refusal exactly in that case. *(by prop:monoid)*

□

Proof of 2.0.18. The Domain struct holds a Law¡Obligation¿ attached to its typed domain D; proposing calls judge then admit in sequence, and an Andon::Halted on either step returns the Err branch with its denial word; the manufacturing step is unreachable from the Err branch, so $\operatorname{im} \mu_{\mathrm{prop}}$ is contained in Admitted objects. *(by def:obsauth)*

□

Proof of 2.0.19. Fan-in and body caps are enforced by admit_rule, so proof nodes have bounded out-degree; the receipt commits the proof by its root hash, and replay recomputes the root from the stored decision, agreeing iff untampered; the verifier reads the fixed-width root, not the DAG, so its cost is $O(\kappa)$. *(by def:logicadm)*

□

Proof of 2.0.20. Bitwise OR on u64 is associative, commutative, idempotent, with identity the zero word; compose inherits these, so $\mathbb{D}$ is a bounded commutative idempotent monoid whose bottom is ADMITTED, and $\text{is\_admitted}(d) \iff d.0 = 0 \iff d = \mathsf{ADMITTED}$. *(by def:denialcode)* Each named constant c_i $(1 \le i \le 7)$ occupies lane i alone, so for any subset $S \subseteq \{1, \dots, 7\}$ the composite $\bigvee_{i \in S} c_i$ has byte lane i nonzero iff $i \in S$; the map $\bigvee_{i \in S} c_i \mapsto \mathbf{1}_S \in \mathbb{D}_7$ is a well-defined bijection $\langle L \rangle \to \mathbb{D}_7$ carrying compose to $\vee$ and ADMITTED to $\mathbf{0}$: a monoid isomorphism. *(by prop:monoid)*

□

Proof of 2.0.21. run_kernel_query constructs RefusalScenario::KernelDenied on the Denied branch and RefusalScenario::KernelInvalid on the Invalid branch; their categories are as stated by thm:total, and denial_lane sends both to CONFORMANCE_GATE_FAILED; being elements of $\mathbb{D}$, they are acted on by all the structure of the preceding chapters. *(by thm:total)*

□

Proof of 2.0.22. π_f is a homomorphism and Φ_o is a homomorphism; their composite carries compose to $\vee$, so $\pi_f(\bigvee_i \varphi_o(s_i)) = \bigvee_i \pi_f(\varphi_o(s_i))$. *(by thm:firedhom)*
The pass folds $\vee$ over N bytes; by cor:allstages applied fleet-wide, the fold is $\mathbf{0}$ iff each $b^{(r)} = \mathbf{0}$ iff each aggregate is ADMITTED iff every agent is admitted, and packing eight bytes per machine word makes the fold word-parallel. *(by cor:allstages)*
□

Proof of 2.0.23. Disjunction is associative, commutative, and idempotent per coordinate, with unit 0 per coordinate; a finite product of such structures inherits all four laws, giving a bounded commutative idempotent monoid; the map $d \mapsto \operatorname{supp} d$ carries $\vee$ to $\cup$ and $\mathbf{0}$ to $\varnothing$, and is a bijection $\mathbb{D}_n \to 2^{[n]}$, hence a monoid isomorphism onto $(2^{[n]}, \cup, \varnothing)$. *(by def:denialabstract)*
In any join-semilattice the identity $d \preceq d' \iff d \vee d' = d'$ holds by antisymmetry of $\subseteq$; least/greatest elements are $\varnothing$ and $[n]$, i.e. $\mathbf{0}$ and $\mathbf{1}$. *(by prop:monoid)*
□

Proof of 2.0.24. A Rust match on a closed enum type-checks only if every constructor is covered or a wildcard is present; the cited match covers all three constructors and uses no wildcard, so the compiler certifies exhaustiveness, and each arm returns a specific RefusalScenario, so scn is a total function. *(by def:ob)*
□

Proof of 2.0.25. Rust's exhaustiveness checker operates on closed sum types; a newtype over u64 exposes 2^{64} inhabitants with no finite constructor set, so no wildcard-free match on its value is well-typed — the implementation is necessarily an if/else chain over the eight known constants with a None fallback, returning None on ADMITTED, Some on each c_i, and None on any other word, including every $c_i \vee c_j$ with $i \neq j$. *(by def:decoder)*
□

Proof of 2.0.26. lane matches all thirteen constructors with no wildcard, hence is total, by the same mechanism as thm:total; for each of the seven denial-lane scenarios s, $\mathrm{lane}(s) = c_i$ is its own lane, and $\mathrm{scn}_\ell(c_i) = s$, giving $\mathrm{scn}_\ell(\mathrm{lane}(s)) = s$; conversely $\mathrm{lane}(\mathrm{scn}_\ell(c_i)) = \mathrm{lane}(s) = c_i$. *(by thm:total)*
Both round-trips are the identity on their respective seven-element sets; the six non-lane scenarios map by lane onto lanes already claimed by the seven, so lane is not injective overall and those six are outside the bijection. *(by def:decoder)*
□

Proof of 2.0.27. $\pi_f(\mathsf{ADMITTED}) = \mathbf{0}$ since every lane of the zero word is zero; byte lane j of $\mathrm{compose}(a, b) = a.0 \mid b.0$ is $a_j \mid b_j$, and for byte values $\mathbf{1}[a_j \mid b_j \neq 0] = \mathbf{1}[a_j \neq 0] \vee \mathbf{1}[b_j \neq 0]$, so π_f satisfies the claimed homomorphism law lane by lane. *(by def:fired)*
On $\langle L \rangle$, prop:code_is_sl identifies elements with subsets $S \subseteq \{1, \ldots, 7\}$, and π_f sends such an element to the indicator of S in bits $1 \ldots 7$; this is a bijection onto $\{0\} \times \{0,1\}^7$ preserving join and bottom. *(by prop:code_is_sl)*

□

Proof of 2.0.28. $\Phi_o(\varepsilon) = \mathsf{ADMITTED}$ and $\Phi_o(w \cdot w') = \bigvee_{s \in w} \varphi_o(s) \vee \bigvee_{s \in w'} \varphi_o(s) = \Phi_o(w) \vee \Phi_o(w') = \mathrm{compose}(\Phi_o(w), \Phi_o(w'))$ by associativity of $\vee$; so Φ_o is a monoid homomorphism agreeing with φ_o on length-one words, and uniqueness is the universal property of the free monoid. *(by def:pipeline)*
Commutativity gives order-independence ($a \vee b = b \vee a$) and idempotence gives multiplicity-independence ($a \vee a = a$), so $\Phi_o(w) = \bigvee_{s \in \mathrm{set}(w)} \varphi_o(s)$ depends only on the underlying set of distinct

denials. *(by prop:monoid)*

□

Proof of 2.0.29. $d_{G'}(o) = \bigvee_{g \in G'} \delta_g(o) = \big(\bigvee_{g \in G} \delta_g(o)\big) \vee \big(\bigvee_{g \in G' \setminus G} \delta_g(o)\big) = d_G(o) \vee d_{G' \setminus G}(o) \succeq d_G(o)$, since in a join-semilattice $x \preceq x \vee y$ always. *(by prop:monoid)*
If $o \in \mathcal{O}^*_{G'}$ then $d_{G'}(o) = \mathsf{ADMITTED}$, the bottom; from $d_G(o) \preceq d_{G'}(o) = \mathsf{ADMITTED}$ and minimality of $\mathsf{ADMITTED}$ we get $d_G(o) = \mathsf{ADMITTED}$, i.e. $o \in \mathcal{O}^*_G$. *(by prop:bottom)*

□

Proof of 2.0.30. Independence means the feasibility of account a's targets is unaffected by the targets of any $b \neq a$; hence the joint optimization $\max_{\text{joint}} \sum_a r(a)$ decomposes into $\sum_a \max_{s_a} r(a, s_a)$ by linearity of the sum and independence of the maxima. *(by def:mrr)*

□

Proof of 2.0.31. A Rust match on the closed RefusalScenario enum type-checks only under exhaustiveness; the category method matches all thirteen constructors with no wildcard arm, so the compiler certifies that every scenario is assigned, and each arm returns a single RefusalCategory, making cat total and single-valued. *(by def:tax)*
Reading the thirteen arms exhibits a preimage for each of the seven listed buckets, so all seven are in the image; and no arm returns Reserved, so $\text{cat}^{-1}(\text{Reserved}) = \varnothing$, hence im cat is precisely the seven-element complement of {Reserved}. *(by def:tax)*

□

Proof of 2.0.32. Kernel::query's first action is the admission gate: if admit_atom(q) returns Err(c) the kernel returns Invalid(c) immediately, before any search, establishing case (c) and its no-proof clause. *(by def:logicadm)*
Phase 1 scans fact blocks for rows unifying with q; a nonempty match set is assembled into Answered decisions, each with a positive proof rooted at the matched fact hash; Phase 2, on empty Phase 1, applies each rule whose head unifies with q by depth-one backward chaining, giving case (a); the fan-in bound holds because admit_rule rejects any node exceeding fan-in 8. *(by def:queryresult)*
Phase 3 is reached iff both searches are empty; the kernel then assembles a Denied decision with a negative proof recording the exhausted search, giving case (b); the three phases are mutually exclusive by construction, so exactly one branch fires. *(by def:queryresult)*
Soundness and completeness of the depth-bounded resolution — that Phase 3 is reached only when q is underivable within the fragment — is SLD soundness and completeness for definite/stratified Horn programs, restricted to the finite search space the byte caps of def:logicadm guarantee. *(by def:logicadm)*

□

Chapter 3

Receipt Cryptography

Axiom 3.0.1 (ax:cr)**.** H *(BLAKE3) is collision-resistant: no PPT adversary finds* $x \neq y$ *with* $\mathsf{H}(x) = \mathsf{H}(y)$ *except with negligible probability* $\varepsilon(\lambda)$*,* $\lambda = 256$*, birthday bound* $\sim 2^{128}$*.*

Construction 3.0.2 (con:commit)**.** *Let* $p \in \{0,1\}^*$ *be the canonical JSON bytes of an admitted payload and* $\mathrm{dg}(p) = \mathsf{H}(p) \in \{0,1\}^{256}$ *its digest; partition* $\mathrm{dg}(p)$ *into eight* 32*-bit little-endian words* $w_0, \ldots, w_7$ *and set* `obj_refs`$[i]$ = `PackedObjRef`(w_i) *via the raw tuple constructor, preserving all* 32 *bits.*

Construction 3.0.3 (con:ocel)**.** *The map* $\Omega : \mathcal{L}\lceil\} \to$ OCEL2.0 *sends each frame to an OCEL event with event id* `instruction_id`*, activity the label of* `activity_idx`*, timestamp* `ts_ns`*, and E2O qualifiers to* `object_ids`*.*

Construction 3.0.4 (con:provo)**.** *Each receipt maps to a PROV-O shape: the admitted observation a* `prov:Entity`*, the manufacture a* `prov:Activity` *that used it, the artifact a* `prov:Entity` *generated by the activity, and the chain link a* `prov:wasDerivedFrom` *edge, SHACL-validated against* `sr:SharedReceiptV1`*.*

Corollary 3.0.5 (cor:antibody)**.** *A claim of the form "this artifact is good because its receipt verifies" is an overclaim: it asserts a virtue Theorem 3.0.33 shows is not entailed. The admissible claim is the narrow conjunction of integrity, attribution, and conformance-as-checked.*

Corollary 3.0.6 (cor:cost)**.** *A verifier checking a single frame recomputes one fold and one equality test:* $O(1)$ *time and space, independent of* n*; checking the boundary projection of a whole run is* $O(\kappa)$*.*

Corollary 3.0.7 (cor:orphan)**.** *No actuated artifact lacks a chain position (no orphan) and no chain position is shared by two actuations (no double-count); consequence is neither created without an admitted cause nor recorded without a unique receipt slot.*

Definition 3.0.8 (def:body)**.** The hash body $\beta(\mathsf{fr}) \in \{0,1\}^{8\cdot 99}$ is the 99-byte serialization produced by `to_hash_bytes`; it is a total, injective function of the semantic fields of fr (all fields except the structural `pad`).

Definition 3.0.9 (def:chain)**.** For a frame fr with predecessor commitment h_-, the successor commitment is $h_+ = \mathsf{H}(h_- \,\|\, \beta(\mathsf{fr}))$; a chain $\mathcal{L}\lceil\} = (\mathsf{fr}_1, \ldots, \mathsf{fr}_n)$ has $h_0 = \mathsf{g}$ and $h_t = \mathsf{H}(h_{t-1} \,\|\, \beta(\mathsf{fr}_t))$, terminal value h_n.

Definition 3.0.10 (def:contentaddr)**.** The address of a stored object b is `content_address`$(b) = \mathsf{H}(b)$ in hex; two byte-identical objects share an address, any change yields a different address.

Definition 3.0.11 (def:fitness)**.** Replay fitness is $\varphi = 1 - \frac{\text{tokens forced on unenabled transitions}}{\text{tokens the replay attempted}} \in [0,1]$, represented as Q16.16 with unit $\texttt{0x0001\_0000} = 65536$; the `token_replay` stage accepts a record iff its replayed fitness equals that unit.

Definition 3.0.12 (def:frame)**.** A frame fr is the record $\langle \texttt{instruction\_id}, \texttt{fired\_mask}, \texttt{denial}, \texttt{obj\_refs}[0{:}8], \texttt{ts\_ns}$ laid out as a `repr(C, align(64))` structure occupying exactly 128 bytes, with 5 bytes of interior padding.

Definition 3.0.13 (def:genesis)**.** The genesis chain value is $\mathsf{g} = \mathsf{H}(\mathbf{0}_{32})$; a run may additionally bind a domain seed $\mathsf{g}_{\text{dom}} = \mathsf{H}(\text{project-version-genesis})$ so chains from distinct projects or versions cannot cross-verify.

Definition 3.0.14 (def:gitlock)**.** An atomic CAS lock for resource L at repository HEAD OID H is acquired by `git update-ref refs/locks/`L H $\mathbf{0}$, which fails if `refs/locks/`L already exists and succeeds atomically otherwise via POSIX rename; deletion on drop releases the lock.

Definition 3.0.15 (def:gitlock-op)**.** For named resource L at HEAD OID H, the CAS lock is acquired by `git update-ref refs/locks/`L H $\mathbf{0}_{20}$, atomically creating `refs/locks/`L pointing to H iff the reference is absent; deletion on drop releases the lock.

Definition 3.0.16 (def:pipeline)**.** The validator runs five stages in fixed order over a ledger $(\mathsf{fr}_t)_{t=1}^{n}$ – `schema`, `chain_recompute`, `chain_linkage`, `monotonic`, `token_replay` – each returning Pass/Fail/Skip, reporting all stage outcomes rather than short-circuiting.

Definition 3.0.17 (def:signed)**.** A signed receipt is the triple $\texttt{SignedReceipt} = \langle h_{\text{hex}}, s, k \rangle$ where $s = \text{Sign}_{sk}(h_{\text{hex}})$ is an Ed25519 signature and k the hex verifying key; it is self-contained since s and k travel with h_{hex}.

Proposition 3.0.18 (prop:bodyinj)**.** *β is injective on the semantic field tuple: if $\beta(\mathsf{fr}) = \beta(\mathsf{fr}')$ then fr and fr' agree on every field of Definition 3.0.12.*

Proposition 3.0.19 (prop:cas)**.** *In a content-addressed lake, storing the same receipt twice is idempotent, and mutating a stored receipt changes its address, so the old address still resolves to the old bytes: history is immutable by construction.*

Proposition 3.0.20 (prop:failclosed)**.** *When the `signed` feature is enabled, a missing or unreadable signing key is a hard error (`SigningFailed`), not a silent skip.*

Proposition 3.0.21 (prop:fitness)**.** *A record's replay attains $\varphi =$ `0x0001_0000` iff its lifecycle is a genuine firing sequence of the fixed POWL token model; otherwise the first forced consumption localizes the nonconformant step, and the stage fails naming that record.*

Proposition 3.0.22 (prop:fold)**.** *h_n is a deterministic function of $(\mathsf{g}, \mathsf{fr}_1, \ldots, \mathsf{fr}_n)$, computable by a single left fold; equal inputs give bit-identical h_n.*

Proposition 3.0.23 (prop:intauth)**.** *Integrity (`verify_chain_hash`) checks s against the embedded key k; authenticity (`verify_chain_hash_with_key`) checks s against an externally pinned key $k^{\star}$. The first is necessary but not sufficient for the second.*

Proposition 3.0.24 (prop:mapcommit)**.** *Ω and the PROV-O bridge are lossless over the committed fields: the terminal h_n is recoverable from the exported log's per-event fields, so exporting does not weaken the chain guarantee.*

Proposition 3.0.25 (prop:nodrift)**.** *A receipt emitted by* `LawObject::receipt_with_record` *and later recomputed by* `ReceiptRecord::recompute_chain_hash` *both route through* `build_admission_frame` *and* `chain_from_frame`*; for identical stored fields they compute identical h_+, so any disagreement is attributable to a changed stored field, never divergent code paths.*

Proposition 3.0.26 (prop:notesledger)**.** *The audit ledger is stored under* `refs/notes/praxis/audit`*; appending entries is a commit transition writing hash-chain receipts as NDJSON notes, preserving immutability of the execution timeline.*

Proposition 3.0.27 (prop:notesledger-op)**.** *Under the CAS lock of Definition 3.0.15, each append of a receipt record is a commit transition serialized as NDJSON under the annotated commit OID, immutable by Git's content-addressed object model, with concurrent appends serialized by the acquire-fail branch of the CAS.*

Proposition 3.0.28 (prop:payloadcommit)**.** *Under Construction 3.0.2, $\beta(\mathsf{fr})$ determines $\mathrm{dg}(p)$, and by Proposition 3.0.18 any change to $\mathrm{dg}(p)$ changes $\beta(\mathsf{fr})$: a frame commits to the content of the artifact, not an opaque handle.*

Proposition 3.0.29 (prop:prefix)**.** *For each t, h_t commits every frame $\mathsf{fr}_1, \ldots, \mathsf{fr}_t$ and the genesis value; the terminal h_n commits the entire history.*

Theorem 3.0.30 (thm:conservation)**.** *Fix $\mathcal{A} = \mu(\mathcal{O}^*)$ with the receipt chain of Definition 3.0.9. Every actuated artifact is caused by an admitted $x \neq \bot$; each actuation is positioned by exactly one appended frame, with $\Phi : \text{actuation} \mapsto h_+$ injective up to collision; each committed frame is attributed at least one admitted cause via bytes $[24, 56)$ and $[16, 24)$.*

Theorem 3.0.31 (thm:faithful)**.** *Let $\mathcal{L}\lceil\rceil = (\mathsf{fr}_1, \ldots, \mathsf{fr}_n)$ have terminal commitment h_n, and $\mathcal{L}\lceil\rceil'$ differ in at least one committed field of at least one frame; if $\mathcal{L}\lceil\rceil'$ has the same terminal h_n, a collision of H has been exhibited. Under Axiom 3.0.1, altering any committed field while preserving h_n succeeds only with probability $\leq \varepsilon(\lambda)$.*

Theorem 3.0.32 (thm:localize)**.** *Suppose a lawful ledger $\mathcal{L}\lceil\rceil$ is perturbed to $\mathcal{L}\lceil\rceil'$ with first affected record index j. The pipeline of Definition 3.0.16 reports a Fail whose named record index is j, and the stage identifier names the class of fault; the honest prefix $\mathsf{fr}_1, \ldots, \mathsf{fr}_{j-1}$ is not implicated.*

Theorem 3.0.33 (thm:scope)**.** *Let a receipt chain $\mathcal{L}\lceil\rceil$ verify and optionally be authentically signed. Then $\mathcal{L}\lceil\rceil$ establishes integrity, attribution, and conformance-as-checked, and no more: it does not establish obligation adequacy, world-fitness, or semantic truth.*

Proof of 3.0.5. The broad claim entails world-fitness (b), which the Scope Theorem shows is independent of the verdict, so the broad claim is not supported by the receipt; the narrow claim is exactly the conjunction (A)∧(B)∧(C), each proven. *(by thm:scope)*

□

Proof of 3.0.6. The fold reads two fixed-width operands and produces a 256-bit output, constant work; equality on 256-bit values is constant; the boundary projection has $\leq \kappa$ chunks by construction, so applying the boundary predicate is $O(\kappa)$. Neither cost references n or the interior dimension. *(by thm:faithful)*

□

Proof of 3.0.18. Each field occupies a disjoint, fixed byte range and each integer field's little-endian encoding is a bijection on its width; a byte-equal body forces range-wise, hence field-wise, equality. Pad bytes are absent from the body so frames differing only in padding correctly have equal bodies. *(by def:body)*

□

Proof of 3.0.19. The address is $\mathsf{H}(b)$, a function of b; equal b gives equal address. *(by def:contentaddr)*
A change $b \to b'$ with $b \neq b'$ gives $\mathsf{H}(b') \neq \mathsf{H}(b)$ except on a collision, so the reference $\mathsf{H}(b)$ cannot be made to resolve to b'. *(by ax:cr)*

□

Proof of 3.0.20. The `signed` feature exists to guarantee a signature; silently skipping would defeat its purpose and let an unsigned receipt masquerade as covered, so key absence is treated as failure: authenticity is either present and checkable or explicitly absent, never ambiguously assumed. *(by def:signed)*

□

Proof of 3.0.21. Enabled firings force no consumption, giving numerator 0 and $\varphi = 1$; a disabled event forces a consumption, making the numerator positive and $\varphi < 1$ at the earliest offending index. The stage compares against the exact fixed-point unit. *(by def:fitness)*

□

Proof of 3.0.22. Each step of the chain rule is a pure function of two arguments, and composition of pure functions is pure; BLAKE3 is a fixed function, so no nondeterminism enters. *(by def:chain)*

□

Proof of 3.0.23. `verify_chain_hash` instantiates the key argument with the embedded k; any adversary who re-signs a modified h with a self-generated key k' passes this check, so it does not establish authorship. *(by def:signed)*
`verify_chain_hash_with_key` instantiates the key argument with $k^\star$; by Ed25519's existential unforgeability, producing a valid s over h under $k^\star$ without $sk^\star$ is infeasible, so a pass attributes authorship to the holder of $sk^\star$. *(by def:signed)*

□

Proof of 3.0.24. Ω carries each frame's committed fields into event attributes; the chain recomputation depends only on these plus `payload_hash` and `prev_chain_hash`, which the record retains, so the exported log determines h_n and re-verification is the Faithful Chain Theorem applied to the reconstructed frames. *(by prop:nodrift)*

□

Proof of 3.0.25. There is a single construction site, `build_admission_frame` and `chain_from_frame`, invoked by both callers on the record's own fields; function purity forces equal outputs on equal inputs, so a recompute mismatch isolates a field mutation rather than a logic difference. *(by prop:fold)*

□

Proof of 3.0.27. A failing CAS (reference already exists) exits with a non-zero code before touching the notes tree; only the lock holder proceeds. *(by def:gitlock-op)*
If the process is killed after the notes append but before lock release, the lock entry remains; the

restart protocol detects a stale lock by comparing the locked OID H to current HEAD, force-releasing it if they differ, since the locked OID no longer uniquely identifies a live critical section. *(by def:gitlock-op)*

□

Proof of 3.0.28. Bytes $[24, 56)$ of the body are the eight words $w_0 \dots w_7 = \mathrm{dg}(p)$ by construction; the map $\mathrm{dg}(p) \mapsto (w_0, \dots, w_7)$ is a little-endian bijection $\{0,1\}^{256} \to (\{0,1\}^{32})^8$, so injectivity of β gives that distinct payload digests induce distinct bodies. *(by prop:bodyinj)*

□

Proof of 3.0.29. Base case: $h_0 = \mathsf{g}$. *(by def:genesis)*
Inductive step: $h_t = \mathsf{H}(h_{t-1} \parallel \beta(\mathsf{fr}_t))$ depends on h_{t-1}, which by hypothesis commits $\mathsf{fr}_1, \dots, \mathsf{fr}_{t-1}$ and g, and on $\beta(\mathsf{fr}_t)$; so h_t depends on all of $\mathsf{g}, \mathsf{fr}_1, \dots, \mathsf{fr}_t$. The collision clause is deferred to the Faithful Chain Theorem. *(by def:chain)*

□

Proof of 6.0.18. Actuation is gated by the equation: an artifact enters $\mathcal{A}$ only as $\mu(x)$ with $x \neq \bot$, so $\mathrm{im}\,\mu$ exhausts the actuated set; `receipt` is a method available only on `LawObject<_, Admitted, _>`, so no artifact is receipted without an admitted antecedent. *(by def:chain)*
Each call to `chain` increments `frame_count` by one and produces one new `chain_hash`. Suppose two distinct actuations mapped to the same h_+: their frames differ (distinct `instruction_id` or payload digest), so equal terminal commitments would force a collision by the Faithful Chain Theorem; hence Φ is injective up to $\varepsilon(\lambda)$. *(by thm:faithful)*
By Construction 3.0.2, $\mathrm{dg}(x)$ occupies bytes $[24, 56)$; the denial word occupies $[16, 24)$ and equals ADMITTED on the accept path, so the frame commits both the identity of the cause and the verdict that admitted it. *(by con:commit)*

□

Proof of 6.0.19. Let j be the least index at which $\mathcal{L}\lceil\}, \mathcal{L}\lceil\}'$ differ in a committed field. Bodies are injective on committed fields, so $\beta(\mathsf{fr}_j) \neq \beta(\mathsf{fr}'_j)$; because j is least, the predecessors agree, $h_{j-1} = h'_{j-1}$. *(by prop:bodyinj)*
The two fold inputs $x = h_{j-1} \parallel \beta(\mathsf{fr}_j)$ and $x' = h'_{j-1} \parallel \beta(\mathsf{fr}'_j)$ have equal-length, equal prefixes but differing suffixes, so $x \neq x'$; Construction 3.0.2's payload-digest word placement is what makes the suffix differ when only the payload changes. *(by prop:payloadcommit)*
If $h_j \neq h'_j$, Proposition 3.0.29 propagates the divergence to the terminal, so $h_n = h'_n$ requires some later fold step to map unequal inputs to equal outputs; if instead $h_j = h'_j$ despite $x \neq x'$, $\mathsf{H}(x) = \mathsf{H}(x')$ is itself a collision. Either way $h_n = h'_n$ forces a collision, contradicting the collision-resistance axiom up to $\varepsilon(\lambda)$. *(by ax:cr)*

□

Proof of 3.0.32. If the mutation malforms a hex field, `schema` fails at j, since earlier records parse, being unperturbed. *(by def:pipeline)*
If the mutation changes a committed field but leaves `chain_hash_hex`(j) stale, `chain_recompute` fails at j: the recomputation uses the perturbed fields and diverges from the stored value exactly at j, since records $< j$ are unperturbed and recompute equal. *(by prop:nodrift)*
If the mutation reorders or splices records so thread continuity breaks, `chain_linkage` fails at the least index whose `prev` no longer matches its predecessor's `chain_hash`. *(by def:pipeline)*
If the mutation rewrites both a field and its stored chain hash consistently, either linkage breaks at j unless the entire suffix is rewritten, in which case the forged terminal commitment differs from

the attested h_n and the Faithful Chain Theorem applies at the boundary. *(by thm:faithful)* If the mutation preserves hashes but produces a trace that is not a firing sequence, `token_replay` fails at j with fitness < 1. In every class the least failing index is j, and the honest prefix passes every stage because each stage's predicate on records $< j$ is evaluated on unperturbed data and holds by hypothesis. *(by def:pipeline)*
□

Proof of 3.0.33. (A) Integrity is the Faithful Chain Theorem: altering a committed field forces a collision. *(by thm:faithful)*
(B) Attribution is Conservation of Consequence clause (iii) composed with the integrity-vs-authenticity distinction: the frame commits the admitted cause and denial word, and (if signed) the signature commits a keyholder. *(by thm:conservation)*
(C) Conformance-as-checked is the fitness proposition, scoped to the fixed model: fitness certifies conformance to the model that was replayed, not that the model is the correct model. *(by prop:fitness)*
(a) Two ledgers with different obligation sets $G \neq G'$ can both verify; the chain commits $\mathrm{dg}(G)$ so it records which set was used, but verification returns Pass for any committed G, so it does not rank G against G'. *(by thm:scope)*
(b) Fix any admitted x; both a benign and a harmful deterministic μ produce verifying chains, since verification never inspects the world-consequences of $a = \mu(x)$, only that a's digest is committed. Hence world-fitness is not a function of the verdict. *(by thm:scope)*
(c) By the series' Rice specialization, no computable predicate decides whether o means what it purports; admission is a retraction, not a decision, so a verifying receipt attests admission, provably weaker than truth. *(by thm:scope)*
□

Chapter 4

Planning Geometry

Construction 4.0.1 (con:join)**.** *Grounding with type constraints is a relational join: for a schema parameter v of type τ, the admissible objects are $\{o : \texttt{TypeIndex::satisfies}(\mathrm{SymId}(o), \mathrm{SymId}(\tau))\}$, walking the parent chain to depth bounded by the height of $\mathcal{T}$, reducing effective branching from N^k to $\prod_i |\{o : \mathrm{type}(o) \le \tau_i\}|$.*

Construction 4.0.2 (con:normalize)**.** *Given a partition part $T' \subseteq T$ with entry places P_{in} and exit places P_{out}, the sub-net is normalized to a valid WF-net by adding fresh boundary places p_s, p_e and silent transitions τ_{in}, τ_{out}, redirecting boundary arcs onto a unified source and sink.*

Construction 4.0.3 (con:strips)**.** *Take the places to be the ground atoms; a ground action t with delete-effects $\boldsymbol{m}_t^-$ and add-effects $\boldsymbol{m}_t^+$ is a transition, and `GroundProblem::find_plan` searches marking space by BFS with the successor relation exactly transition firing.*

Construction 4.0.4 (con:tape)**.** `temporal_plan_to_powl_tape` *converts a* `TemporalPlan` *into a tape of* `PowlOpSpecs`*: each step i becomes an activity with a* `pred_mask` *of steps $j < i$ finishing before i starts and a one-hot* `succ_mask`*; a second pass performs transitive reduction, clearing bits of predecessors' own predecessors, leaving only direct predecessors.*

Construction 4.0.5 (con:xorf)**.** *Let $R \subseteq \mathcal{O}$ be the reachability fixpoint fact set. A frozen fact store builds an 8-bit XOR filter over FNV-1a hashes of R, with index mapping via the fastrange reduction* $\mathrm{reduce}(x, n) = (x \times n)/2^{32}$*; membership is gated by $x \in B$ if $F(x) = \mathrm{true}$, else false, with false-positive rate $\approx 0.4\%$ and false negatives* 0*.*

Corollary 4.0.6 (cor:cert)**.** *A nonconformance certificate is a single vector $\boldsymbol{y} \in \mathbb{R}^p$ together with $\mathbf{N}^\top \boldsymbol{y} \le \mathbf{0}$ and $\boldsymbol{y}^\top \boldsymbol{b} > 0$; checking it is a matrix product and two sign tests, independent of trace length, verifiable within a bounded context κ.*

Corollary 4.0.7 (cor:dominate)**.** *An admitted plan sorts strictly before any refused plan regardless of its secondary costs: for any finite risk, makespan, token, latency, and switch values, an admitted* `CostVector` *is $\prec_{\mathrm{lex}}$ the refused top element; the price of unlawfulness is infinite in the limit that defines the order.*

Definition 4.0.8 (def:balance)**.** A durative action j draws on a fluent ϕ if its effect decreases ϕ by r_j at start and increases it by r_j at end for rate $r_j \ge 0$, holding r_j units over $[s_j, e_j)$; the free level of ϕ at time t is $f_\phi(t) = \nu_0(\phi) - \sum_{j : t \in [s_j, e_j)} r_j$.

Definition 4.0.9 (def:conf)**.** A trace with final marking $\boldsymbol{m}^\dagger$ and transition-count vector $\boldsymbol{x}$ conforms to a net with initial marking $\boldsymbol{m}_0$ if $\boldsymbol{m}^\dagger \in \mathcal{P}$ and the $\boldsymbol{x}$ witnessing it is realized by a genuine firing ordering; it fails membership if $\boldsymbol{m}^\dagger \notin \mathcal{P}$.

Definition 4.0.10 (def:cost)**.** The router's `CostVector` is the tuple $\boldsymbol{c} = (c_0, \ldots, c_5) = (\overline{\texttt{admitted}}, \texttt{risk}, \texttt{attention\_s}$ the order is lexicographic, cheapest-first, and the refused vector $\texttt{refused}() = (\text{unadmitted}, 255, \infty, \infty, \infty, 255)$ is the top element.

Definition 4.0.11 (def:domain)**.** A lifted domain is a tuple $\mathcal{D} = (\mathcal{T}, \mathcal{P}, \mathcal{F}, \mathcal{S})$: a finite type hierarchy $\mathcal{T}$, a finite set $\mathcal{P}$ of predicate symbols, a finite set $\mathcal{F}$ of numeric function symbols (fluents), and a finite set $\mathcal{S}$ of action schemas; a durative schema carries typed parameters, a duration constraint, a condition C_s, and an effect E_s.

Definition 4.0.12 (def:fitness)**.** On finalization the verifier returns, in Q16.16 fixed point, $\varphi = |\texttt{fitted}|/|\texttt{replayed}|$ and precision $= |\texttt{replayed}|/(|\texttt{replayed}| + |\texttt{enabled\_not\_taken}|)$, where $|\cdot|$ is population count and $\varphi \coloneqq 1$ when the denominator is zero.

Definition 4.0.13 (def:ground)**.** Given a domain $\mathcal{D}$ and a problem $(\mathcal{O}\lfloor, \boldsymbol{m}_0, \nu_0, \gamma)$, the grounding substitutes every type-compatible object tuple into each schema's parameters, producing a finite set of ground actions; a grounded state is a pair $(\boldsymbol{m}, \nu)$ of a set $\boldsymbol{m}$ of true ground atoms and a fluent valuation ν.

Definition 4.0.14 (def:incidence)**.** Let $\mathbf{N} \in \mathbb{Z}^{p \times |T|}$ have column $\boldsymbol{\delta}_t = \boldsymbol{m}_t^+ - \boldsymbol{m}_t^-$ for each transition t; a firing sequence with Parikh vector $\boldsymbol{x} \in \mathbb{Z}_{\geq 0}^{|T|}$ satisfies the state equation $\boldsymbol{m} = \boldsymbol{m}_0 + \mathbf{N}\boldsymbol{x}$.

Definition 4.0.15 (def:makespan)**.** Given the precedence DAG with per-op durations d_i and release times, the forward pass computes $\text{es}_i = \max(r_i, \max_{j \prec i} \text{ef}_j)$, $\text{ef}_i = \text{es}_i + d_i$; the makespan is $T = \max_i \text{ef}_i$; the backward pass computes $\text{ls}_i = \text{lf}_i - d_i$; the slack of op i is $\text{ls}_i - \text{es}_i$, and the critical path is the set of zero-slack ops.

Definition 4.0.16 (def:net)**.** A net has p places and a finite set T of transitions; a marking is a vector $\boldsymbol{m} \in \mathbb{Z}_{\geq 0}^p$; a transition t is a pair $(\boldsymbol{m}_t^-, \boldsymbol{m}_t^+)$ of preset and postset, enabled at $\boldsymbol{m}$ iff $\boldsymbol{m} \geq \boldsymbol{m}_t^-$ coordinatewise, firing to $\boldsymbol{m}' = \boldsymbol{m} - \boldsymbol{m}_t^- + \boldsymbol{m}_t^+$.

Definition 4.0.17 (def:polytope)**.** The marking polytope of a net is the relaxation $\mathcal{P} = \{\boldsymbol{m}_0 + \mathbf{N}\boldsymbol{x} : \boldsymbol{x} \geq \mathbf{0}\} \cap \{\boldsymbol{m} \geq \mathbf{0}\} \subseteq \mathbb{R}_{\geq 0}^p$, the continuous translated cone whose integer points over-approximate the reachable set.

Definition 4.0.18 (def:powl)**.** A POWL model over an activity alphabet A is one of: an activity $a \in A$ (a leaf); a partial order $(\{M_1, \ldots, M_k\}, \prec)$ executed by respecting $\prec$ and running incomparable children concurrently; or a choice graph $(\{M_1, \ldots, M_k\}, E)$ combined by exclusive choice (and cyclic/loop edges), taking exactly one live branch.

Definition 4.0.19 (def:replay)**.** The verifier holds a marking $\boldsymbol{m}$ (`enabled_tokens`) initialized to the entry token, and accumulators `replayed`, `fitted`, `enabled_not_taken`; replaying a frame rejects an invalid node bit, requires $\boldsymbol{m} \geq \boldsymbol{m}^-$, records enabled-but-unconsumed tokens, fires $\boldsymbol{m} \leftarrow (\boldsymbol{m} \;\&\; \neg\boldsymbol{m}^-) \mid \boldsymbol{m}^+$, and sets the node bit in `replayed` and `fitted`.

Definition 4.0.20 (def:saturation)**.** Let $(\mathcal{D}, \mathcal{O}\lfloor)$ be a grounded PDDL problem with initial atom set $\boldsymbol{m}_0$ and ground action set T. The Nemo Datalog program Π has one fact per atom in $\boldsymbol{m}_0$ and one rule per ground action; the saturation $\text{sat}(\Pi)$ is the least fixpoint $\text{lfp}(T_\Pi)$ of the immediate-consequence operator; an atom p is reachable iff $p \in \text{sat}(\Pi)$.

Definition 4.0.21 (def:symdict)**.** A symbol identifier SymId $\in \mathbb{Z}_{>0}$ is a compact integer encoding of a ground atom or object name; `pddl-index` maintains a bidirectional map `SymDict` : {strings} $\leftrightarrow \mathbb{Z}_{>0}$, encoded forward as `FxHashMap<String,u32>` and reverse as `Vec<String>`; zero is reserved as undefined.

Proposition 4.0.22 (prop:balance)**.** *A durative-action schedule is feasible with respect to ϕ iff $f_\phi(t) \geq 0$ for all t; for the unit-rate attention fluent this says the number of concurrently in-flight capabilities never exceeds the capacity $\nu_0(\texttt{attention})$.*

Proposition 4.0.23 (prop:cone)**.** *Every marking $\boldsymbol{m}$ reachable from $\boldsymbol{m}_0$ satisfies the state equation for some $\boldsymbol{x} \in \mathbb{Z}_{\geq 0}^{|T|}$, hence lies in the integer points of $\boldsymbol{m}_0 + \mathrm{cone}(\mathbf{N})$ intersected with $\mathbb{Z}_{\geq 0}^{p}$; the state equation is necessary for reachability but not sufficient.*

Proposition 4.0.24 (prop:conserve)**.** *If every action drawing on ϕ both decrements it by r_j at start and increments it by r_j at end, the net effect of each completed action on ϕ is zero, and the terminal valuation equals the initial: $\nu_{end}(\phi) = \nu_0(\phi)$; scheduling redistributes when ϕ is held, never how much exists.*

Proposition 4.0.25 (prop:critical)**.** *The makespan T equals the length of the longest ($\prec$-)path in the precedence DAG weighted by durations, and an op has zero slack iff it lies on some longest path; delaying a zero-slack op delays the makespan, delaying an op by less than its slack does not.*

Proposition 4.0.26 (prop:dictcost)**.** *With symbol dictionary encoding, a ground atom $p(o_1, \ldots, o_k)$ is a tuple of $k+1$* SymId *values; equality reduces to integer comparison and hash-set membership to a single hash over $4(k+1)$ bytes, so the grounding loop's cost with encoding replaces $O(|T| \cdot k \cdot N)$ string comparisons with $O(1)$-time integer counterparts.*

Proposition 4.0.27 (prop:hasse)**.** *Let $\prec$ be the strict order "j finishes before i starts" on plan steps. The initialized `pred_mask` of step i is $\{j : j \prec i\}$, and after transitive reduction it is the covering relation of $\prec$; the resulting dependency graph is a DAG, and two steps are schedulable concurrently iff $\prec$-incomparable.*

Proposition 4.0.28 (prop:honest)**.** *φ and precision are ratios of bit-populations of disjointly-maintained bitsets; neither can exceed 1, and $\varphi = 1$ is attained exactly on completed replays; `enabled_not_taken` feeds precision, not fitness, so the two metrics measure orthogonal deviations.*

Proposition 4.0.29 (prop:local)**.** *In a POWL model, the token marking local to a node ranges over the tokens of that node's immediate children: a node of arity k has a local marking space of dimension $O(k)$, independent of the total size of the model; a node's replay is checked in a working set bounded by its arity.*

Proposition 4.0.30 (prop:quarantine)**.** *Let sep be the decidable predicate "the net is sound and separable." It is a legitimate admission obligation: total and computable, retracting the space of arbitrary control-flow onto the decidable sub-language of POWL-expressible processes; a separable net is admitted, a non-separable net is refused with reason.*

Proposition 4.0.31 (prop:safe)**.** *On a safe (1-bounded) net, the integer firing rule $\boldsymbol{m}' = \boldsymbol{m} - \boldsymbol{m}_t^- + \boldsymbol{m}_t^+$ coincides with the branchless bitset update $\texttt{enabled\_tokens} \leftarrow (\texttt{enabled\_tokens} \,\&\, \neg \boldsymbol{m}_t^-) \mid \boldsymbol{m}_t^+$, and the enabling test coincides with the branchless subset check $(\texttt{enabled} \,\&\, \boldsymbol{m}_t^-) \oplus \boldsymbol{m}_t^- = 0$.*

Proposition 4.0.32 (prop:satcorrect)**.** *$p \in \mathrm{sat}(\Pi)$ iff there exists a sequence of ground actions whose add-effects include p, i.e. p is reachable from $\boldsymbol{m}_0$ in the Petri-net sense.*

Proposition 4.0.33 (prop:sensitivity)**.** *Let $T(\phi = c)$ be the makespan the planner finds when fluent ϕ's initial capacity is c. The capacity delta reports $(T(c-1), T(c), T(c+1))$, and ϕ is binding iff $T(c-1) \neq T(c)$ (or the problem is infeasible at $c-1$); this is a finite-difference sensitivity of the specific schedule the greedy planner produced, not a dual shadow price of an optimal scheduler.*

Theorem 4.0.34 (thm:bounded-ground)**.** *Let $\mathcal{D}$ have schemas of maximum parameter arity k over $|\mathcal{O}\rfloor| = N$ objects. The number of ground actions is at most $|\mathcal{S}| \cdot N^k$, a finite constant independent of the goal; consequently a forward search over grounded actions has a branching factor bounded a priori, and μ's manufacture terminates within a fixed depth bound.*

Theorem 4.0.35 (thm:farkas)**.** *Fix $\boldsymbol{b} = \boldsymbol{m}^\dagger - \boldsymbol{m}_0$. Exactly one holds: (a) there exists $\boldsymbol{x} \geq \mathbf{0}$ with $\mathbf{N}\boldsymbol{x} = \boldsymbol{b}$; or (b) there exists $\boldsymbol{y} \in \mathbb{R}^p$ with $\mathbf{N}^\top \boldsymbol{y} \leq \mathbf{0}$ and $\boldsymbol{y}^\top \boldsymbol{b} > 0$, a separating hyperplane certifying $\boldsymbol{m}^\dagger \notin \boldsymbol{m}_0 + \mathrm{cone}(\mathbf{N})$, and if (b) holds no firing sequence reaches $\boldsymbol{m}^\dagger$.*

Theorem 4.0.36 (thm:fitness)**.** *Replaying a sequence of frames against the net either (i) completes with no violation, in which case it is a genuine firing sequence and $\varphi = 1$; or (ii) halts at the first frame whose preset is not contained in the current marking, returning a coordinate-localized witness of the earliest disabled transition.*

Theorem 4.0.37 (thm:lang-correct)**.** *Let N be a safe and sound workflow net, and let $\psi = \mathit{convert}(N)$ be its POWL 2.0 conversion. Then for any prefix trace length k, $\mathcal{L}_k(N) = \mathcal{L}_k(\psi) = \mathcal{L}_k(\mathit{recompose}(\psi))$.*

Theorem 4.0.38 (thm:lex)**.** *Let secondary coordinates be bounded, $|c_i| \leq M$ for $i \geq 1$. Define $C_\lambda(\boldsymbol{c}) = \sum_{i=0}^{5} \lambda^{5-i} c_i$ with $\lambda > 1$. There is a threshold Λ such that for all $\lambda > \Lambda$, $\boldsymbol{c} \preceq_{\mathrm{lex}} \boldsymbol{c}' \iff C_\lambda(\boldsymbol{c}) \leq C_\lambda(\boldsymbol{c}')$; as $\lambda \to \infty$ the weight on c_0 diverges relative to all others.*

Theorem 4.0.39 (thm:mrr)**.** *Let A be a set of independent client accounts, $\mathit{realized}(a)$ the revenue of account a under a target stage, and $\mathit{lawful\_targets}(a)$ its evidence-gated target stages. The joint plan optimization decomposes linearly: $\max_{\mathit{joint\ plan}} \sum_{a \in A} \mathit{realized}(a) = \sum_{a \in A} \max_{t \in \mathit{lawful\_targets}(a)} \mathit{realized}(a, t)$, reducing search complexity from $\Omega(\prod_a |T_a|)$ to $O(|A| \cdot |T_a|)$.*

Theorem 4.0.40 (thm:sep)**.** *A sound, separable workflow net admits a recursive decomposition into a POWL model whose every internal node is a partial order or a choice graph and whose leaves are activities, with each node's local marking geometry dimension bounded by its arity, independent of the global net size.*

Proof of 4.0.6. Verification reads $\boldsymbol{y}$ and $\boldsymbol{b}$ and evaluates $\mathbf{N}^\top \boldsymbol{y}$ and $\boldsymbol{y}^\top \boldsymbol{b}$; the cost is $O(p \cdot |T|)$ in the net's fixed dimensions, with no dependence on trace length or interior computation. *(by thm:farkas)*
□

Proof of 4.0.7. The lead coordinate c_0 is 0 for admitted and 1 for refused, and by Theorem 6.0.21 it dominates: any admitted vector differs from the refused element first at c_0, where it is strictly smaller. *(by thm:lex)*
□

Proof of 4.0.22. The start guard at s_j requires $\nu(\phi) \geq r_j$ in the state just before j's at-start decrement, so the guard holds iff $f_\phi(s_j) \geq 0$; free level changes only at action starts and ends, so nonnegativity at every start implies it everywhere, and a violated guard is an instant with $f_\phi < 0$. *(by def:balance)*

□

Proof of 4.0.23. Firing t once adds column $\boldsymbol{\delta}_t$ to the marking; a sequence firing each t exactly x_t times adds $\sum_t x_t \boldsymbol{\delta}_t = \mathbf{N}\boldsymbol{x}$, giving the state equation, and since each $x_t \geq 0$ and markings are nonnegative integers, $\boldsymbol{m}$ is an integer point of $\boldsymbol{m}_0 + \mathrm{cone}(\mathbf{N})$. *(by def:net)*
Insufficiency is the classical Petri-net fact: the state equation ignores intermediate nonnegativity and firing order, so spurious solutions exist. *(by prop:cone)*
□

Proof of 4.0.24. `Decrease`(ϕ, r_j) then `Increase`(ϕ, r_j) compose to the identity on $\nu(\phi)$; summing over all completed actions, the total change is $\sum_j (r_j - r_j) = 0$, so $\nu_{\text{end}}(\phi) = \nu_0(\phi)$. *(by def:balance)*
□

Proof of 4.0.25. The forward recursion $\text{ef}_i = \text{es}_i + d_i$ with $\text{es}_i = \max_{j \prec i} \text{ef}_j$ is the standard longest-path dynamic program over a DAG, so $T = \max_i \text{ef}_i$ is the overall longest path; the backward pass gives the latest each op may start without increasing T, and slack 0 iff i lies on a longest path. *(by def:makespan)*

□

Proof of 4.0.26. Each parameter binding substitutes an object's SymId (an $O(1)$ lookup in a `Vec`) for a parameter name; forming the atom tuple is $O(k)$ integer writes, and the hash of a tuple of 32-bit integers is computed by FxHash in $O(k)$ time independent of string lengths. *(by def:symdict)* Summing over $|T|$ schemas and N^k bindings gives the stated cost. *(by thm:bounded-ground)*
□

Proof of 4.0.27. $\prec$ is irreflexive and transitive, induced by the total order on real finish/start times, hence a strict partial order; the first pass sets bit j in i's mask exactly when $j \prec i$. *(by con:tape)* The reduction clears bit j from i's mask when some retained predecessor k of i has j in its predecessors, i.e. $j \prec k \prec i$; what survives is precisely the covering relation (the Hasse diagram) of $\prec$. *(by con:tape)*
□

Proof of 4.0.28. Population count is monotone and bounded by the number of node bits, and `fitted`$\subseteq$`replayed` by construction, so $\varphi \leq 1$. *(by def:replay)*
Equality holds iff no frame was replayed-without-fitting, which is every completed replay; `enabled_not_taken` is updated only in step (3) and read only by precision's denominator, so it is independent of the fitness numerator. *(by thm:fitness)*
□

Proof of 4.0.29. A node's execution semantics reference only its own children's tokens: a partial order gates on the $\prec$-predecessors among its k children, a choice gates on which of its k branches is live, so the token bits touched by one node's firing are contained in an $O(k)$-dimensional subspace. *(by def:powl)*

□

Proof of 4.0.30. Soundness and separability are structural properties of a finite net, checkable by the decomposition procedure, which either returns a POWL tree or the node at which decomposition fails; thus sep is total and computable. *(by thm:sep)*
Admission maps the net to its POWL decomposition when sep $= 1$ and to refusal with the obstruction as refusal data otherwise, exactly the admission map specialized to processes. *(by thm:sep)*
□

Proof of 4.0.31. On 0/1 markings with $\boldsymbol{m}_t^- \leq \boldsymbol{m}$, the AND-NOT clears exactly the consumed places and the OR with $\boldsymbol{m}_t^+$ sets the produced places; when preset and postset are disjoint from the retained marking this is bit-for-bit the integer expression. *(by def:net)*
The enabling identity holds because $(\boldsymbol{m} \,\&\, \boldsymbol{m}^-) \oplus \boldsymbol{m}^- = 0$ iff every bit of $\boldsymbol{m}^-$ is also set in $\boldsymbol{m}$, i.e. $\boldsymbol{m} \geq \boldsymbol{m}^-$ coordinatewise on 0/1 vectors. *(by con:strips)*
□

Proof of 4.0.32. Soundness: every derived atom is the add-effect of a ground action whose preconditions were derived earlier, by induction on the derivation depth; the actions form a witness firing sequence. *(by def:saturation)*
Completeness: a reachable atom has a witness sequence; unrolling it gives a derivation in T_Π, the standard Datalog fixpoint theorem applied to the grounded action rules. *(by prop:satcorrect)*
□

Proof of 4.0.33. `replan_with_perturbed_capacity` re-solves `find_temporal_plan` with ϕ's initial value moved by ± 1 and reads off the resulting makespan; `binding_resource_mask` sets bit i iff the -1 perturbation changed the makespan or made the problem infeasible. *(by def:makespan)*
Infeasibility at $c - 1$ is itself evidence the resource binds. *(by prop:balance)*
□

Proof of 4.0.34. Each schema binds at most k parameters, each to one of $\leq N$ objects, so it grounds to at most N^k actions; summing over $|\mathcal{S}|$ schemas gives the bound. *(by def:ground)*
In `bcinr-pddl` the count is additionally capped by `PDDL8_MAX_GROUND`, exceeding it is `Pddl8Error::BoundExceeded`, so the ground action set is finite and its size is a fixed function of $(|\mathcal{S}|, N, k)$, discharging boundedness (M2). *(by def:ground)*
□

Proof of 4.0.35. This is Farkas' lemma applied to the system $\mathbf{N}\boldsymbol{x} = \boldsymbol{b},\ \boldsymbol{x} \geq \mathbf{0}$: either the system has a nonnegative solution, or there is $\boldsymbol{y}$ with $\boldsymbol{y}^\top \mathbf{N} \leq \mathbf{0}^\top$ and $\boldsymbol{y}^\top \boldsymbol{b} > 0$, and never both. *(by thm:farkas)*
In case (b), if a firing sequence reached $\boldsymbol{m}^\dagger$, its Parikh vector $\boldsymbol{x}^\star \geq \mathbf{0}$ satisfies $\mathbf{N}\boldsymbol{x}^\star = \boldsymbol{b}$, so $\boldsymbol{y}^\top \boldsymbol{b} = (\mathbf{N}^\top \boldsymbol{y})^\top \boldsymbol{x}^\star \leq 0$, contradicting $\boldsymbol{y}^\top \boldsymbol{b} > 0$; hence (b) rules out reachability. *(by prop:cone)*
Soundness is one-directional: feasibility of the relaxation (a) is necessary but not sufficient for an actual firing sequence, so (a) does not by itself certify conformance — only (b) certifies its negation. *(by prop:cone)*

□

Proof of 4.0.36. Step (2) of the replay definition admits a frame iff $\boldsymbol{m} \geq \boldsymbol{m}^-$, which is exactly the enabling condition; by induction on the prefix, if no frame is rejected then every frame fired from a marking at which it was enabled, so the whole sequence is a genuine firing sequence. *(by def:replay)*
Step (4) is the firing rule of the safe-net specialization. *(by prop:safe)*
In the completing case every replayed frame also sets `fitted`, so $|\texttt{fitted}| = |\texttt{replayed}|$ and $\varphi = 1$; conversely the guard fails at the first frame whose required tokens are absent, returning immediately with that frame's node id, the localized witness. *(by def:fitness)*
□

Proof of 6.0.21. If $\boldsymbol{c} \prec_{\text{lex}} \boldsymbol{c}'$ they first differ at coordinate p with $c'_p - c_p \geq \epsilon > 0$; then $C_\lambda(\boldsymbol{c}') - C_\lambda(\boldsymbol{c}) \geq \lambda^{5-p}\epsilon - 2M(\lambda^{5-p} - 1)/(\lambda - 1)$, positive once $\lambda > \Lambda := 1 + 2M \cdot 6/\epsilon$, so the scalarization order agrees with $\preceq_{\text{lex}}$ for $\lambda > \Lambda$. *(by def:cost)*
□

Proof of 6.0.23. This is the decomposition established for POWL by Kourani and van der Aalst and colleagues, resting on van der Aalst's process-mining foundations; cited rather than reproved here. *(by thm:sep)*
□

Chapter 5

Projection and Scale

Construction 5.0.1 (con:agent8)**.** *Project each agent a to a status byte $\mathbf{b}(a) \in \{0,1\}^8$ whose lanes are the OK-polarity complements of the denial lanes (admitted, evidence, budget, authority, healthy, conformant, receipted, replayable); $\mathbf{1} =$ `0xFF` is the all-OK byte; an agent is admissible iff $\mathbf{b}(a) = \mathbf{1}$, equivalently iff its 8-lane denial restriction is $\mathbf{0}$.*

Construction 5.0.2 (con:swar)**.** *For a denial word $w \in \{0,1\}^{64}$ of eight 8-bit lanes with $L_{high} =$ `0x8080808080808080` and $L_{low7} =$ `0x7f7f7f7f7f7f7f7f`, the zero-lane bitmask is $z(w) = \neg(((w\&L_{low7})+ L_{low7}) \vee w)\&L_{high}$; bit 7 of lane j in $z(w)$ is 1 iff lane j in w is exactly `0x00`.*

Corollary 5.0.3 (cor:diag)**.** *Under the Mutant Kill Theorem, the index of the killing stage certifies which law-bearing invariant a mutation broke; a non-empty surviving set that is not equivalent-mutants is a proof that the frame under-commits, the Faithful Projection converse.*

Corollary 5.0.4 (cor:onedenial)**.** *In denial polarity, with $D_\ell = \neg P_\ell$ the 64-agent word of denial lane ℓ, the denied set is $\bigvee_\ell D_\ell$ and the admitted set its complement; fleet admission is the word-parallel join of denial lanes followed by one complement, and adding a ninth lane costs one more AND (resp. OR) per 64 agents.*

Corollary 5.0.5 (cor:onlybit)**.** *A planetary control plane that re-admits its population continuously cannot be built on comprehension: no attainable accelerator fleet closes the demand gap; it can be built on the sweep, whose cost is dominated by a single pass over* 10 GB *of memory; comprehension is reserved for sampled audit at honest $O(n)$ replay cost.*

Corollary 5.0.6 (cor:reflexive)**.** *Each quantitative claim in this paper is either a theorem with a proof or an Estimate explicitly labelled with its inputs; the partition is exhaustive, so this paper's overclaim set is empty, and by the magnitude theorem the magnitude of each claim is bounded by its witness as the invariant requires.*

Corollary 5.0.7 (cor:throughput)**.** *A single commodity server admits $N = 10^{10}$ agents in $\sim$ 25–*100 ms*, i.e. at $\sim 10^{11}$–4×10^{11} agent-admission decisions per second per server, bandwidth-bound.*

Definition 5.0.8 (def:astgate)**.** An AST gate is a syntactic predicate $g : \mathcal{T} \to \{0,1\}$ on an abstract syntax tree $\mathcal{T}$, computable in $O(|\mathcal{T}|)$ time; the retrofit applier gates every proposed change through an ordered battery $(g_1, \ldots, g_m)$: the change is admitted iff all gates pass, $\bigwedge_i g_i(\mathcal{T}) = 1$; a failing gate returns a denial with the gate index and offending AST node.

Definition 5.0.9 (def:claim)**.** A claim is a pair $c = (\phi_c, w_c)$ where ϕ_c is a proposition and $w_c \in \mathbb{R}_{\geq 0}$ its asserted magnitude; a receipt witness for c is a receipt $r(c)$ whose boundary projection is accepted,

$V(\pi(r(c))) = 1$, and whose committed fields entail an attested magnitude $\widehat{w}_c$; a claim is admissible iff it has a receipt witness with $w_c \le \widehat{w}_c$.

Definition 5.0.10 (def:diff)**.** For $f, g : D \to R$ targeting the same S, the differential oracle is $\Omega_{f,g}(x) = [f(x) = g(x)]$; a test passes at x iff $\Omega_{f,g}(x) = 1$.

Definition 5.0.11 (def:fuzz)**.** For the total admission retraction $\alpha : \mathcal{O} \rightharpoonup \mathcal{O}^* \cup \{\bot\}$, the fuzzing oracle Ω_∂ maps generated $o \in \mathcal{O}$ to 1 iff $\alpha(o)$ terminates and yields either an admitted $x \in \mathcal{O}^*$ with well-formed artifact $\mu(x)$ or a refusal $\bot$ carrying a category in the eight-bucket taxonomy; $\Omega_\partial(o) = 0$ iff α crashes, diverges, or returns an unlabelled outcome.

Definition 5.0.12 (def:instance-q)**.** For an executed manufacture σ: $T(\sigma)$, the interior token count (measured, a count of logged records); κ, the boundary field count of the receipt projection $\pi(\sigma)$ (a design constant); t_{H}, the per-frame recomputation wall-time (estimated from published hash throughput, not measured here).

Definition 5.0.13 (def:layouts)**.** The array-of-structures (AoS) layout stores each agent's 8-bit byte contiguously, a 64-bit word holding 8 agents' bytes; the structure-of-arrays (SoA / bit-plane) layout stores each lane ℓ as its own bitset $P_\ell \in \{0,1\}^N$, a 64-bit word of P_ℓ holding the lane-ℓ bit of 64 agents.

Definition 5.0.14 (def:mut)**.** A mutation operator m maps a valid subject $s^\star \in \bigcap_i I_i$ to a mutant $m(s^\star)$ violating a non-empty set of invariants; its correct stage is $\mathsf{s}(m) = \min\{i : m(s^\star) \notin I_i\}$; m is killed by V, written $\mathrm{Kill}(m) = 1$, iff V rejects $m(s^\star)$.

Definition 5.0.15 (def:oracle)**.** For implementation $f : D \to R$ and specification $S \subseteq D \times R$, the correctness question is $Q_S(f) \equiv \forall x\,(x, f(x)) \in S$; a decidable oracle for f on finite $X \subseteq D$ is a total computable predicate $\Omega : X \to \{0,1\}$ intended to witness $(x, f(x)) \in S$, terminating on every $x \in X$.

Definition 5.0.16 (def:over)**.** The overclaim set of a system emitting claims $\mathcal{C}$ is $\mathsf{Over} = \{c \in \mathcal{C} : \nexists\, r(c)$ with $V(\pi(r(c))) = 1$ and $w_c \le \widehat{w}_c\}$; the antibody invariant is $\mathsf{Over} = \varnothing$, the `docs-exceed-mechanism` defect class enforced as a gate.

Definition 5.0.17 (def:residual)**.** For environmental observations O and target artifact A, the measurement $\mu(O)$ returns a residual vector $R \in \mathbb{R}^k$ with $r_i = \mathrm{measured}_i - \mathrm{midpoint}(\mathrm{target}_i)$; reconciliation selects the dominant dimension $\arg\max_i |r_i|$ and applies a repair operator subject to `RepairBand` limits.

Definition 5.0.18 (def:staged)**.** A staged validator is a pipeline $V = (V_1, \ldots, V_k)$ where each stage $V_i : \mathcal{S} \to \{\mathrm{pass}\} \cup \{\mathrm{reject}_i\}$ tests a decidable invariant $I_i \subseteq \mathcal{S}$; the pipeline accepts s iff $s \in \bigcap_i I_i$, reporting on rejection the least i with $s \notin I_i$. Stage V_i is sound if $V_i(s) = \mathrm{reject}_i \Rightarrow s \notin I_i$ and complete if $s \notin I_i \Rightarrow V_i(s) = \mathrm{reject}_i$.

Definition 5.0.19 (def:vizgap)**.** A visual gap report is the output of `measure_gap`: for each reconcilable dimension $i \in \{1, \ldots, k\}$, the residual $r_i = \mathrm{measured}_i - \mathrm{midpoint}(\mathrm{target}_i)$ and the dominant dimension $i^\star = \arg\max_i |r_i|$ together with a rendered diff block; the report has k residual values and one dominant index, size $O(k)$ independent of the interior.

Definition 5.0.20 (def:worktree)**.** An applier isolates modifications by spawning a decoupled working tree via `git worktree add temp_path phase_branch`, running validation gates and executing the auto-rollback protocol `git reset --hard baseline_sha` on failure to guarantee main branch cleanliness.

Definition 5.0.21 (def:worktree-ret)**.** The retrofit applier isolates each application attempt in a fresh Git worktree, applies the change, runs the CI oracle $(g_1, \ldots, g_m)$ (`cargo build`, `cargo test`, `cargo clippy`), and on any gate failure executes `git reset --hard baseline_sha`; on all-pass the worktree change is merged and the chain receipt for the application is minted with the worktree OID committed into the frame's `obj_refs` payload digest.

Estimate 5.0.22 (est:bit-supply)**.** *One commodity server supplies $\sim 10^{11}$ decisions/s bandwidth-bound; a rack of $\sim 10^2$ servers supplies $\sim 10^{13}$ decisions/s, $\sim 10^7\times$ cheaper per decision than an LLM judgment, so bit-parallel supply scales into the demand band on a single facility.*

Estimate 5.0.23 (est:comp-supply)**.** *A comprehension-based admission decision costs $\sim 10^{-2}$–10^{0} s of accelerator time, i.e. ~ 1–10^2 decisions/s per accelerator; a planetary fleet of $\sim 10^6$–10^7 accelerators yields a comprehension-supply ceiling $\Lambda_{comp} \lesssim 10^7$–$10^9$ decisions/s, central estimate $\sim 10^8$ decisions/s.*

Estimate 5.0.24 (est:demand)**.** *For a population $N \in [10^9, 10^{12}]$ agents each re-admitted at rate $\rho \in [1, 10^2]\,\mathrm{s}^{-1}$, aggregate admission demand is $\Lambda_{demand} = N\rho \in [10^9, 10^{14}]$ decisions/s, central estimate $\sim 10^{11}$–10^{12} decisions/s for $N \sim 10^{10}$, $\rho \sim 10$.*

Estimate 5.0.25 (est:sweep)**.** *A full admission sweep reads all 8 planes once ($10\,\mathrm{GB}$) and performs $7N/64 \approx 1.1 \times 10^9$ AND instructions for $N = 10^{10}$; at streaming bandwidth $B \sim 10^2$–$4 \times 10^2\,\mathrm{GB/s}$, $t_{sweep} \approx 10\,\mathrm{GB}/B \approx 25$–$100\,\mathrm{ms}$; the instruction count is exact, the wall-time is an order-of-magnitude estimate.*

Estimate 5.0.26 (est:wall)**.** *At BLAKE3 throughput 1–3 GB/s/core and a few-hundred-byte frame, $t_{\mathsf{H}} \sim 10^{-7}$ s/frame; a four-field boundary compare is $\ll 10^{-7}$ s; per-message verification with $c \sim 10$ spot frames costs $\sim 10^{-6}$ s; these are order-of-magnitude estimates, the constancy-in-T point is the proved Proposition 5.0.31.*

Proposition 5.0.27 (prop:astterm)**.** *For any finite AST $\mathcal{T}$ and finite gate battery $(g_1, \ldots, g_m)$ of syntactic predicates, battery evaluation terminates in $O(m \cdot |\mathcal{T}|)$ time and returns either `Admitted` or a refusal carrying the first failing gate index and offending node; the retraction is total, never diverging and never returning an unlabelled outcome.*

Proposition 5.0.28 (prop:diff-bound)**.** *Under independent errors at rates p_f, p_g with wrong outputs drawn from $\geq M$ distinguishable values with collision probability $\leq 1/M$: $\Pr[\Omega_{f,g}(x) = 1 \wedge (x, f(x)) \notin S] \leq p_f p_g / M$; over a corpus of size n the probability some silent-agreement error survives all n tests is at most np_fp_g/M, and a fixed silently-wrong input missed with per-input hit rate q has miss probability $(1-q)^n \to 0$.*

Proposition 5.0.29 (prop:footprint)**.** *A population of $N = 10^{10}$ agents, each an 8-bit byte, occupies $N \times 8$ bits $= N$ bytes $= 10^{10}$ bytes $= 10\,\mathrm{GB}$ in either layout; this is an exact arithmetic identity, not an estimate.*

Proposition 5.0.30 (prop:fuzz)**.** *If α is total and terminating with codomain $\mathcal{O}^* \cup \{\bot\}$ and $\bot$ is refusal-with-category, then for every $o \in \mathcal{O}$, $\Omega_\partial(o) = 1$, and any observed $\Omega_\partial(o) = 0$ is a genuine defect in the retraction, not a property of the input.*

Proposition 5.0.31 (prop:invariance)**.** *For manufactures $\{\sigma_i\}$ sharing the frame schema with $T(\sigma_i) \to \infty$, $\mathrm{Ver}_\partial(\sigma_i) = \Theta(\kappa)$ is constant in i, and the estimated per-message wall-time $\kappa \cdot t_{cmp} + c \cdot t_{\mathsf{H}}$ does not depend on $T(\sigma_i)$.*

Proposition 5.0.32 (prop:mainclean)**.** *Under the isolated worktree applier, the main branch is never advanced past a failing CI gate; the rollback protocol is unconditional on failure, running before the worktree is removed, restoring the branch to* `baseline_sha` *regardless of which gate failed; hence the main branch satisfies all CI gates at every point in its history.*

Proposition 5.0.33 (prop:necessity)**.** *For S a non-trivial semantic property, $Q_S(f)$ is undecidable; hence no procedure decides $Q_S(f)$ for arbitrary f, and any mechanical trust in f must be trust in some Ω evaluated on finite X, plus an argument bounding residual risk on $D \setminus X$.*

Proposition 5.0.34 (prop:vizgap)**.** *The visual gap report gives* $\mathrm{gap} : \mathbb{R}^k \to \mathbb{R}^k \times [k]$*, a projection with the same cost structure as the receipt projection, $O(k)$ to compute and $O(k)$ to verify; the repair operator acts on the dominant dimension $i^\star$ subject to* `RepairBand` *bounds, producing at most one corrective actuation, a bounded deterministic manufacture.*

Theorem 5.0.35 (thm:afford)**.** *For demand Λ_{demand} and supplies $\Lambda_{comp}, \Lambda_{bit}$: $\Lambda_{comp} \sim 10^8 \ll \Lambda_{demand} \sim 10^{11}$–$10^{12} \lesssim \Lambda_{bit} \sim 10^{11}$–$10^{13}$; comprehension-based admission is under-provisioned by three to four orders of magnitude for a 10^{10}-agent fleet, while bit-parallel admission meets demand on a single facility, and the separation persists under an order-of-magnitude perturbation of any single input.*

Theorem 5.0.36 (thm:branchless)**.** *Under the SoA layout, the admissibility of 64 agents is computed by the single branchless word expression* $\mathsf{adm} = P_1 \& \cdots \& P_8$*; bit j of* adm *is 1 iff agent j has all eight lanes OK; the computation uses exactly 7 AND instructions, no data-dependent branch, and processes 64 agents, cost* $7/64 \approx 0.11$ *instructions per agent.*

Theorem 5.0.37 (thm:instance-gap)**.** *For comprehension cost* $\mathrm{Comp}(\sigma) = \Theta(T(\sigma))$ *and boundary-verification cost* $\mathrm{Ver}_\partial(\sigma) = \Theta(\kappa)$*, the instance ratio is* $\Theta(T(\sigma)/\kappa) = \Theta(2.5 \times 10^6/4) \approx 6.25 \times 10^5$*; both inputs are measured/design constants, so the ratio is a measured consequence, not an estimate.*

Theorem 5.0.38 (thm:kill)**.** *For a staged validator $V = (V_1, \ldots, V_k)$ in which every stage is sound and complete, and a mutation operator m with correct stage* $\mathsf{s}(m)$*:* $\mathrm{Kill}(m) = 1 \iff V_{\mathsf{s}(m)}$ *rejects $m(s^\star)$, and the least rejecting stage reported by V equals* $\mathsf{s}(m)$*.*

Theorem 5.0.39 (thm:magnitude)**.** *If a system maintains* $\mathsf{Over} = \varnothing$*, then for every emitted claim c, $w_c \le \widehat{w}_c = \max\{w :$ the committed fields of $r(c)$ entail magnitude $w\}$, verifiable at boundary cost $O(\kappa)$ independent of the interior that produced c; no claim can exceed what its mechanism can receipt.*

Theorem 5.0.40 (thm:swar-verify)**.** *For a 64-bit word w packing 8 agents gated against the broadcast required mask, the number of admitted agents is* $\mathrm{admitted}(w) =$ `popcount`$(z(w))$*, sweeping 8 agents in one CPU instruction with zero branches and no lane borrow leakage.*

Proof of 5.0.6. The gap ratio, the footprint, the per-word gating, and the magnitude bound itself are theorems with proofs. *(by thm:magnitude)*
The wall-time, sweep time, demand, supplies, and affordability separation are labelled estimates carrying their inputs; every magnitude falls in one class or the other, none unlabelled, hence $\mathsf{Over} = \varnothing$ for this document. *(by def:over)*
□

Proof of 5.0.27. Each g_i traverses the finite tree once in $O(|\mathcal{T}|)$ time; the battery evaluates gates in order, returning the structured denial on first failure or `Admitted` on all-pass; each gate result composes as a denial-monoid element. *(by def:astgate)*
□

Proof of 5.0.28. A false accept at x requires f wrong and $g(x) = f(x)$, probability $\leq p_f p_g / M$ by independence and the collision bound; the union bound over n corpus points gives $np_f p_g / M$; the miss bound is the probability a Bernoulli-q trigger never fires in n draws, $(1-q)^n$. *(by def:diff)*
□

Proof of 5.0.29. Eight bits is one byte; N agents is N bytes; 10^{10} bytes is 10 GB; both layouts store the same $8N$ bits, hence the same N bytes; no estimated quantity appears. *(by con:agent8)*
□

Proof of 5.0.30. Totality gives termination on all o; the closed codomain gives, for each o, either $x \in \mathcal{O}^*$ with $\mu(x)$ defined, or $\perp$ with a category, since every refusal path maps through the wildcard-free classifier; thus $\Omega_\partial \equiv 1$ on a correct implementation, and an observed 0 contradicts totality or closedness, witnessing a defect. *(by def:fuzz)*
□

Proof of 5.0.31. The projection has codomain of size $\leq \kappa$ and the verifier reads only $\pi(\sigma_i)$; neither the field count nor the verifier's input size is a function of T; a spot-check recomputes a fixed number c of $O(1)$ frames, so its cost is $c \cdot t_{\mathsf{H}}$, independent of T. *(by def:instance-q)*
□

Proof of 5.0.32. The applier's critical section creates the worktree, applies the change, runs the gate battery, and on failure resets to `baseline_sha` and aborts before the worktree is deleted; the main branch pointer advances only on the pass branch, so the main branch history is a subsequence of attempts that all passed every gate. *(by def:worktree-ret)*
□

Proof of 5.0.33. $Q_S(f)$ quantifies a non-trivial semantic predicate over the language computed by f; by Rice's theorem it is undecidable; a total Ω on finite X always terminates and so is not a decision procedure for Q_S, but a retraction onto the decidable fragment "f passes Ω on X"; trust in f factors through Ω and X. *(by prop:necessity)*
□

Proof of 5.0.34. The residual vector is a linear map costing $O(k)$ arithmetic operations; the argmax is a single pass over k values; the repair acts on that one dimension only, and since `RepairBand` is a finite closed interval, the repair is a bounded projection onto that interval. *(by def:vizgap)*
□

Proof of 5.0.35. The comprehension ceiling $\Lambda_{\text{comp}} \sim 10^8$ falls short of demand $\sim 10^{11}$–10^{12} by 10^3–10^4. *(by est:comp-supply)*
The bit-parallel supply meets or exceeds demand; the gap between the two supplies is the keystone's per-decision cost ratio scaled by population. *(by est:bit-supply)*
A tenfold error in any one input (population, rate, per-decision cost, accelerator count) moves the inequality by at most one decade and does not reverse the three-to-four decade comprehension shortfall. *(by est:demand)*
□

Proof of 5.0.36. Agent a_j is admissible iff $\bigwedge_{\ell=1}^{8} \mathbf{b}(a_j)_\ell = 1$. *(by con:agent8)*
Bit j of P_ℓ equals $\mathbf{b}(a_j)_\ell$ by the SoA definition; bitwise AND acts coordinatewise, so bit j of $P_1 \& \cdots \& P_8$ equals $\bigwedge_\ell \mathbf{b}(a_j)_\ell$, which is 1 iff a_j is admissible; a left-fold of AND over 8 operands is 7 binary ANDs, branchless, giving per-agent cost 7/64. *(by def:layouts)*
□

Proof of 5.0.37. Comprehension must ingest $\Theta(T(\sigma))$ tokens; boundary verification reads the $\kappa = 4$ receipt fields and applies the fixed verifier. *(by def:instance-q)*
The quotient is T/κ; substituting $T \approx 2.5 \times 10^6$, $\kappa = 4$ gives $\approx 6.25 \times 10^5$, immediate from the keystone's Theorem 4.2; no estimated quantity enters. *(by thm:instance-gap)*
□

Proof of 5.0.38. ($\Leftarrow$) If $V_{\mathsf{s}(m)}$ rejects $m(s^\star)$ then V rejects, so $\mathrm{Kill}(m) = 1$. *(by def:mut)*
($\Rightarrow$) If $\mathrm{Kill}(m) = 1$, let j be the least rejecting stage; by soundness of V_j, $m(s^\star) \notin I_j$; by completeness of $V_1, \ldots, V_{j-1}$, none of which rejected, $m(s^\star) \in I_i$ for all $i < j$; hence $j = \mathsf{s}(m)$, and V's reported least rejecting index equals $\mathsf{s}(m)$. *(by def:staged)*
□

Proof of 5.0.39. By $\mathsf{Over} = \varnothing$, every emitted c has a receipt witness with $V(\pi(r(c))) = 1$ and $w_c \le \widehat{w}_c$; a claim with $w_c > \widehat{w}_c$ would lie in Over, contradicting the invariant; hence $w_c \le \widehat{w}_c$, a bound independent of the unread interior. *(by def:over)*
□

Chapter 6

Projection Thesis

Axiom 6.0.1 (ax:obs)**.** *There is a set $\mathcal{O}$, the observation space, whose elements are arbitrary finite records: logs, agent outputs, sensor frames, claims, tool responses. $\mathcal{O}$ carries no decidable semantics: "does this observation mean what it purports to mean" is not assumed computable.*

Construction 6.0.2 (con:agent8)**.** *Project each agent to a status byte $b \in \{0,1\}^8$ whose lanes name admitted, evidence-ok, within-budget, authority-bound, healthy, conformant, receipted, replayable. A fleet of N agents is a bit-packed vector; a 64-bit word holds 8 agents; a fleet admission sweep is the branchless mask reduction of the denial construction applied word-parallel.*

Construction 6.0.3 (con:denial)**.** *Let $D = (\{0,1\}^n, \vee, \mathbf{0})$ be the commutative idempotent monoid of denial words: n independent lanes, componentwise OR, identity $\mathbf{0}$. Each obligation g_i contributes a lane map $d_i : \mathcal{O} \to \{0,1\}^n$ with $d_i(o) = \mathbf{0} \iff g_i(o) = 1$. The total denial is $d(o) = \bigvee_i d_i(o)$, and $\alpha(o) \neq \bot \iff d(o) = \mathbf{0}$.*

Corollary 6.0.4 (cor:noncomp)**.** *An accepting verifier's predicate factors as* Verify $= V \circ \pi$ *for a fixed $V : \mathcal{R}_p \to \{0,1\}$ of input size $\leq \kappa$. Trust in the accept is a function of the κ-sized projection and the soundness of V, not of any agent's comprehension of σ.*

Definition 6.0.5 (def:adm)**.** Fix a decidable $\mathcal{O}^* \subseteq \mathcal{O}$ on which a finite battery of obligations $g_1, \ldots, g_m : \mathcal{O} \to \{0,1\}$ is computable. The admission map is the partial retraction $\alpha : \mathcal{O} \rightharpoonup \mathcal{O}^* \cup \{\bot\}$, $\alpha(o) = \rho(o) \in \mathcal{O}^*$ if $\bigwedge_i g_i(o) = 1$, else $\bot$, with ρ a computable bounding/normalization and $\bot$ the distinguished refusal. Admission is idempotent on its image: $\alpha \circ \alpha = \alpha$.

Definition 6.0.6 (def:attnbalance)**.** Let $c(t) \geq 0$ be capacity and admitted action j draw rate $r_j \geq 0$ on $[s_j, e_j]$. Free capacity is $f(t) = c(t) - \sum_{j:\, t \in [s_j, e_j]} r_j$. A schedule is feasible iff $f(t) \geq 0$ for all t.

Definition 6.0.7 (def:costvector)**.** Assign a plan $\boldsymbol{c} = (c_0, c_1, \ldots, c_k)$, $c_0 \in \{0,1\}$ the unadmitted indicator ($0 =$ admitted), $c_1, \ldots, c_k$ bounded secondary costs. Order by lexicographic $\preceq_{\text{lex}}$.

Definition 6.0.8 (def:fitnessdistance)**.** $\varphi(\sigma) = 1 - \dfrac{\text{tokens consumed on unenabled transitions}}{\text{tokens the replay attempted to consume}} \in [0,1]$; $\varphi = 1$ iff the replay never forced a disabled firing, and $1 - \varphi$ is a normalized ℓ^1 excursion outside the enabled set.

Definition 6.0.9 (def:mu)**.** $\mu : \mathcal{O}^* \to \mathcal{A}$ is a deterministic computable map subject to (M1) determinism/reproducibility: $\mu(x)$ depends only on x, equal admitted inputs give bit-identical artifacts; and (M2) boundedness: μ factors through a representation of size bounded by fixed structural constants, so μ terminates with a priori bounded cost. The Chatman equation is $\mathcal{A} = \mu(\mathcal{O}^*)$, operationally $a = \mu(\alpha(o))$ when $\alpha(o) \neq \bot$, and $\mu(\bot) = \bot$.

Definition 6.0.10 (def:reachcone)**.** Let $N \in \mathbb{Z}^{p\times|T|}$ have columns $\boldsymbol{\delta}_t = \boldsymbol{m}_t^+ - \boldsymbol{m}_t^-$. A firing sequence with Parikh vector $\boldsymbol{x} \geq 0$ reaches $\boldsymbol{m} = \boldsymbol{m}_0 + N\boldsymbol{x}$. The reachable set lies in the integer points of $\boldsymbol{m}_0 + \mathrm{cone}(N)$.

Definition 6.0.11 (def:receipt)**.** Fix a collision-resistant hash $\mathsf{H} : \{0,1\}^* \to \{0,1\}^{256}$ and write $\mathrm{dg}(\cdot) = \mathsf{H}(\cdot)$. For a manufacture with admitted observation $x = \alpha(o)$, obligation set G with denial word $d(o)$, transition identity τ, artifact $a = \mu(x)$, replay result φ, refusal reason r, and implementation version v, the frame is $\mathsf{fr} = \langle \mathrm{dg}(x), \mathrm{dg}(G), d(o), \tau, \mathrm{dg}(a), \varphi, r, v\rangle$, and the chained receipt advances by $h_+ = \mathsf{H}(h_-\|\mathsf{fr})$. The receipt projection is $\pi : \Sigma \to \mathcal{R}_p$, $\pi(\sigma) = (\mathrm{verdict}, h_+, \varphi, r)$, a tuple of at most κ chunks.

Definition 6.0.12 (def:regimes)**.** For a trace of n committed frames: boundary projection inspection (any verifier, per message) costs $O(\kappa)$; single frame recomputation (spot check) costs $O(1)$ per frame; checkpointed/indexed replay (sampled audit) costs $O(\log n)$ or bounded path; full chain replay (full audit) costs $O(n)$.

Proposition 6.0.13 (prop:attnconservation)**.** *Total attention* $\int_0^T \sum_j r_j \mathbf{1}_{[s_j,e_j]}(t)\,dt = \sum_j r_j(e_j - s_j)$ *is independent of ordering. Reordering redistributes* when *attention is spent, not* how much*; the optimization target is the makespan functional* $T[\cdot]$ *under precedence and the pointwise capacity constraint.*

Proposition 6.0.14 (prop:conformembership)**.** *A trace with Parikh vector* $\boldsymbol{x}$ *is conformant only if its marking lies in* $\mathcal{P} = \{\boldsymbol{m}_0 + N\boldsymbol{x} : \boldsymbol{x} \geq \mathbf{0}\} \cap \{\boldsymbol{m} \geq \mathbf{0}\}$*. If* $\boldsymbol{m}^\dagger \notin \mathcal{P}$*, Farkas' lemma yields* $\boldsymbol{y}$ *with* $\boldsymbol{y}^\top N \leq \mathbf{0}$ *and* $\boldsymbol{y}^\top(\boldsymbol{m}^\dagger - \boldsymbol{m}_0) > 0$*: a certificate of nonconformance.*

Proposition 6.0.15 (prop:planetary)**.** *For a fleet of* N *agents under Construction 6.0.2: (a) arithmetic – the status vector occupies exactly* N *bytes, packed to* $N/8$ *64-bit words; a full admission sweep is* $N/8$ *masked-OR word operations. (b) engineering estimate – compressed status is* $\sim 10\,\mathrm{GB}$*; a sweep takes* $\Theta(N/(8B))$ *wall-clock. (c) order-of-magnitude estimate – the affordability ratio between one cache-line mask op and one LLM admission decision is* $\sim 10^7$*. (d) consequence, not theorem – a comprehension-based or per-agent-LLM planetary control plane is infeasible by* $\sim N \cdot 10^7$*, while a bit-parallel admission sweep is feasible.*

Proposition 6.0.16 (prop:selfauditedscale)**.** *Let a system emit claims* c *with receipts* $r(c)$*, and define the overclaim set* $\mathrm{Over} = \{c : \nexists\, r(c) \text{ with } V(\pi) = 1\}$*. If the system enforces* $\mathrm{Over} = \varnothing$ *(the docs-exceed-mechanism defect class), the admissible magnitude of a claim is bounded by the mechanism that receipts its limits, not by rhetoric.*

Proposition 6.0.17 (prop:semilattice)**.** $(\{0,1\}^n, \vee)$ *is the join-semilattice of* $2^{[n]}$ *with bottom* $\mathbf{0}$*. Adding obligations only enlarges denials: for* $G' \supseteq G$*,* $d_{G'}(o) \succeq d_G(o)$*, so* $\{o : \alpha_{G'}(o) \neq \bot\} \subseteq \{o : \alpha_G(o) \neq \bot\}$*.*

Theorem 6.0.18 (thm:conservation)**.** *(i) Causation: every actuated artifact* a *is the image of at least one admitted observation,* $a \in \mathrm{im}\,\mu$*, and no* $a \notin \mathrm{im}\,\mu$ *is actuated. (ii) Position uniqueness: each actuation appends exactly one receipt-chain position; the map actuation* $\mapsto h_+$ *is injective up to hash collision. (iii) Observation recoverability / commitment: if* μ *is injective on* $\mathcal{O}^*$*, the artifact determines the admitted observation (recoverability); otherwise, when the frame commits* $\mathrm{dg}(x)$*, the receipt position uniquely commits to* $(\mathrm{dg}(x), \mathrm{dg}(G), \tau, \mathrm{dg}(a))$ *(commitment, not semantic recovery).*

Theorem 6.0.19 (thm:faithful)**.** *The receipt is faithful to lawful consequence exactly to the extent the frame commits its fields: for any committed field* $\phi \in \{\mathrm{dg}(x), \mathrm{dg}(G), d(o), \tau, \mathrm{dg}(a), \varphi, r, v\}$*, producing*

a trace σ' that differs in ϕ yet matches the claimed h_+ requires a collision of H; under collision resistance a verifier who recomputes the chain rejects any such σ' except with negligible probability. Conversely, fields not committed by the frame are not attested.

Theorem 6.0.20 (thm:gap)**.** *Let $\mathrm{Comp}(\sigma) = \Theta(\dim \Sigma)$ be the cost of comprehending the interior and $\mathrm{Ver}_\partial(\sigma) = O(\kappa)$ the cost of boundary verification of $\pi(\sigma)$. Then $\mathrm{Comp}(\sigma)/\mathrm{Ver}_\partial(\sigma) = \Omega(\dim \Sigma/\kappa) \to \infty$ as $\dim \Sigma \to \infty$, while boundary verification stays constant; replay and full audit honestly cost $O(n)$, reducible to $O(\log n)$ under checkpoint indexing.*

Theorem 6.0.21 (thm:lex)**.** *With $C_\lambda(\boldsymbol{c}) = \sum_{i=0}^{k} \lambda^{k-i} c_i$, $\lambda > 1$, $|c_i| \le M$: there is Λ with $\boldsymbol{c} \preceq_{\mathrm{lex}} \boldsymbol{c}' \iff C_\lambda(\boldsymbol{c}) \le C_\lambda(\boldsymbol{c}')$ for all $\lambda > \Lambda$. As $\lambda \to \infty$ the lawfulness coordinate's weight diverges.*

Theorem 6.0.22 (thm:rice)**.** *Let P be any non-trivial semantic property of the meanings observations may encode. Then $\{o \in \mathcal{O} : P(o)\}$ is undecidable.*

Theorem 6.0.23 (thm:sep)**.** *A sound, separable workflow net admits a recursive decomposition into a POWL 2.0 model whose every node is a choice graph (all exclusive/cyclic logic) or a partial order (all concurrent logic); each node's local marking geometry has dimension bounded by its arity, independent of global net size.*

Theorem 6.0.24 (thm:unitfitness)**.** *A trace is a firing sequence iff $\varphi = 1$. If $\varphi < 1$, the first frame forcing an unenabled consumption is a coordinate-localized witness $t^\star$.*

Proof of 6.0.13. Linearity of the integral; each $r_j(e_j - s_j)$ depends only on j's duration and rate. *(by def:attnbalance)*

□

Proof of 6.0.14. Membership is the state equation with $\boldsymbol{x} \ge 0$; Farkas applied to $N\boldsymbol{x} = \boldsymbol{m}^\dagger - \boldsymbol{m}_0$, $\boldsymbol{x} \ge 0$ gives either a nonnegative solution or the separating $\boldsymbol{y}$. *(by def:reachcone)*

□

Proof of 6.0.15. (a) is counting: the status vector occupies exactly N bytes by construction of the per-agent status byte. *(by con:agent8)*
(b),(c) are estimates and are marked as such; they depend on hardware and workload and are not claimed as theorems. *(by con:agent8)*
(d) is the engineering consequence of composing (a) with the Fleet Trust Budget corollary: per-message cost $O(\kappa)$ versus $\Omega(\dim \Sigma)$ for comprehension, scaled by N. No part of this proposition asserts a mathematical impossibility; it asserts an economic separation under stated estimates. *(by thm:gap)*

□

Proof of 6.0.16. Every admissible claim has a faithful receipt, verifiable at $O(\kappa)$; a claim lacking one is in Over and rejected by the invariant. Admissible claims are exactly receipted claims. *(by thm:gap)*

□

Proof of 6.0.17. Idempotence, commutativity, associativity of $\vee$ per coordinate; products of semilattices are semilattices. *(by con:denial)*
$d_{G'}(o) = d_G(o) \vee \bigvee_{i \in G' \setminus G} d_i(o) \succeq d_G(o)$, and $\mathbf{0}$ is the unique minimum. *(by con:denial)*

□

Proof of 6.0.18. Actuation is gated by the Chatman equation: a actuates only if $a = \mu(x)$, $x = \alpha(o) \neq \bot$; thus $\operatorname{im}\mu$ exhausts the actuated set and its complement never actuates. 'At least one' is all determinism yields, since μ need not be injective. *(by def:mu)*
The chain appends one position per manufacture; distinct positions with equal h_+ force a collision. *(by thm:faithful)*
If μ is injective on $\mathcal{O}^*$, then $a = \mu(x)$ determines x outright – recoverability. Otherwise, because fr contains $\mathrm{dg}(x)$, the frame – hence h_+ – commits the observation digest; by collision resistance no second x' with $\mathrm{dg}(x') \neq \mathrm{dg}(x)$ shares the position, so the receipt uniquely commits to the digest tuple. *(by def:receipt)*
□

Proof of 6.0.19. Each field appears as an argument to H inside fr, and h_- commits all predecessors recursively, so h_+ is a function of every committed field of the entire causal history. *(by def:receipt)*
If σ' differs in a committed ϕ but the recomputed chain equals the claimed h_+, then some application of H took distinct inputs to equal outputs – a collision – which has negligible probability. *(by def:receipt)*
If σ' differs only in a field absent from fr, the chain is unchanged and the difference is, correctly, not attested: the theorem claims faithfulness precisely over committed fields, no more. *(by def:receipt)*
Drop $\mathrm{dg}(x)$ or $d(o)$ from the frame and the same proof yields only byte-tamper-evidence, establishing that lawfulness attestation requires the law-bearing commitments. *(by def:receipt)*
□

Proof of 7.0.27. Ver_∂ reads $\pi(\sigma)$ of size $\leq \kappa$ and applies V: $O(\kappa)$. *(by cor:noncomp)*
Comp must ingest the interior of size $\Theta(\dim \Sigma)$. The ratio diverges. *(by def:regimes)*
The scope clause restates the verification-regimes definition: mechanical replay is $O(n)$, not $O(1)$; the constant-cost claim is confined to boundary inspection. *(by def:regimes)*
□

Proof of 6.0.21. If $\boldsymbol{c} \prec_{\mathrm{lex}} \boldsymbol{c}'$ they first differ at p with $c'_p - c_p \geq \epsilon > 0$; then $C_\lambda(\boldsymbol{c}') - C_\lambda(\boldsymbol{c}) \geq \lambda^{k-p}\epsilon - 2M\sum_{i>p}\lambda^{k-i}$, positive once $\lambda > \Lambda = 1 + 2Mk/\epsilon$. *(by def:costvector)*
Ties map to equality. *(by thm:lex)*
□

Proof of 6.0.22. Rice's theorem: every non-trivial property of the language recognized by a Turing machine is undecidable. Observations are unrestricted encodings and may encode arbitrary machines; any non-trivial semantic predicate inherits undecidability by reduction from the halting problem. *(by ax:obs)*

□

Proof of 6.0.24. Enabled firings never force consumption, giving numerator 0 and, inductively, a genuine firing sequence within $\mathcal{P}$. A disabled event records a forced consumption at the earliest index $t^\star$, making the numerator positive. *(by def:fitnessdistance)*
□

Chapter 7

Synthesis Thesis

Construction 7.0.1 (con:fablechain)**.** *When* $\mathrm{Oracle}(T, R) = 1$*, the harness mints a receipt using the standard* `OcelCausalFrame` *chain:* $h_+ = \mathsf{H}(h_- \,\|\, \mathrm{body}(\mathsf{fr}))$*, where the frame's* `obj_refs` *carry* $\mathrm{dg}(C) = \mathsf{H}(\textit{canonical bytes of } C)$*. The genesis seed is* $\mathsf{H}(\textit{"fable-v1-genesis"})$*, domain-isolated from the plan chain so fable receipts cannot cross-verify with plan receipts even if their terminal hashes collide.*

Construction 7.0.2 (con:merklecell)**.** *Let* $M_{g,i}$ *be the terminal hash of member* i *in group* g*. The Group Receipt hash is* $H_g = \mathsf{H}(\mathit{sort}(\{M_{g,i}\}))$*, and the Cell Receipt hash is the rolling fold over* G *group hashes:* $H_{cell} = \mathsf{H}(H_1 \,\|\, H_2 \,\|\, \ldots \,\|\, H_G)$*.*

Definition 7.0.3 (def:branch)**.** A branch is (c, Σ, r): a class, a conjunction of signal predicates over a crash snapshot, and a lawful response $r \in \{\mathsf{Restart}, \mathsf{Park}(\rho), \mathsf{Refuse}(\mathrm{core}), \mathsf{Escalate}\}$. The geometry Γ maps each node to an ordered branch list; classification is first-match-wins; geometry_hash $= \mathsf{ca}(\Gamma \,\|\, \mathrm{topology\_hash})$.

Definition 7.0.4 (def:classes)**.** FailureClass = {LogicFault, BudgetBreach, AuthorityVacuum, TransientFault, Stall, StarvedInput, CertifiedUnsat, GeometryGap} – the semantic axis, capped at eight by the doctrine.

Definition 7.0.5 (def:depth)**.** Let $G = (V, E)$ be the plan's data-dependency DAG (edge $u \to v$ iff an add-effect of u feeds a precondition of v); define $d(v) = 0$ for sources and $d(v) = 1 + \max_{u \to v} d(u)$ otherwise. A stage $S_k = \{v : d(v) = k\}$.

Definition 7.0.6 (def:earned)**.** Stage S_k supervises RestForOne iff some $v \in S_k$ has a nonempty transitive dependent set $D(v)$; otherwise OneForOne. The cohort of v is $\{v\} \cup D(v)$ under RestForOne and $\{v\}$ under OneForOne.

Definition 7.0.7 (def:fable)**.** A Fable harness is a test environment that constructs a prompt P_T from a task ontology T, calls a mock model membrane M that intercepts the LLM API and returns a deterministic response drawn from a test fixture indexed by P_T's hash, extracts the code block C from the model response R, and runs the verification oracle $\mathrm{Oracle}(T, R)$ against C in an isolated cargo workspace. The mock membrane ensures that harness runs are hermetic: no real LLM call is made, the response is deterministic given P_T, and the oracle verdict is reproducible.

Definition 7.0.8 (def:fragile)**.** A precondition predicate p of capability c is fragile iff no capability in the problem produces p. The initial state is a one-time gift: if a fragile fact is lost mid-run, nothing in the plan can lawfully re-produce it – whereas a fact with even one producer is recoverable by restarting that producer.

Definition 7.0.9 (def:sandbox)**.** Let P_T be a prompt compiled from task ontology T, and let C be the code block extracted from model response R. The sandbox verification oracle is the composite: $\text{Oracle}(T, R) = \text{cargo\_build}(C) \wedge \text{cargo\_test}(C) \wedge \text{cargo\_clippy}(C) \wedge \text{safety\_audit}(C)$. The outcome is receipted and chained to the ledger using rolling BLAKE3 hashes.

Definition 7.0.10 (def:walframe)**.** A cache frame journaling the memoized DAG nodes is serialized as a length-prefixed frame: $\text{WAL\_Frame} = \langle \text{length}, \mathsf{H}(\text{payload}), \text{payload} \rangle$, allowing recovery to discard torn frames.

Lineage 7.0.11 (lineage:actorlineage)**.** *The 8-bounded actor lineage (CNS -¿ BitActor -¿ ByteActor -¿ knhk): CNS fixed the doctrine 8T/8H/8M; ByteActor V3 productized it with 64-byte crystal envelopes and bounded SPSC rings; knhk carried the doctrine into Rust with a branchless TickBudget, a ChatmanBounded assertion, and an R1/W1/C1 runtime-class taxonomy. The lineage's one persistent gap never closed: classification without actuation – nothing in the constellation restarts, re-admits, or closes the loop.*

Lineage 7.0.12 (lineage:armstrong)**.** *Erlang/OTP made three moves that survive every re-examination: processes are isolated so failure cannot spread by memory; supervisors – not the failing code – own the recovery decision; and restart strategies plus a restart intensity turn 'let it crash' into a bounded, structured discipline. What OTP does not do is derive the tree: a human writes the supervision hierarchy, and the crash space lives in the programmer's head.*

Lineage 7.0.13 (lineage:instruments)**.** *The LHC does not record every collision; its trigger system encodes, in advance, the signatures of events worth keeping. The EHT did not photograph a black hole; it reconstructed one from lawful projections gathered by an instrument designed around a predicted signal. The discipline common to both: enumerate the event geometry before the event, instrument for exactly those signatures, and treat anything outside the geometry as a first-class finding – never as noise to be discarded silently.*

Measurement 7.0.14 (meas:cell)**.** *The supervised cell, 10,000 members, release: overhead of supervision at fault rate 0 is* -0.9% *of medians over five paired runs, inside the baseline's own* $\pm 5.7\%$ *run spread – statistical noise. Recovery counts track injected transient rates exactly:* 69/608/3,179 *recovered at* $1\%/10\%/50\%$*. Throughput is flat across all fault rates. The crash-looping template is detected at quorum and quarantined by epoch 2 in every configuration.*

Measurement 7.0.15 (meas:recoveryinvisible)**.** *On the lawobject plan with two injected transient crashes at judge: both crashes land in named branches (TransientFault, then Stall), the run completes, and the final root hash is byte-identical to the crash-free run's. Park-then-re-admit heals to the same identity; machine death composes: kill-9 mid-restart-loop, WAL recovery, identical receipt.*

Proposition 7.0.16 (prop:fable-oracle)**.** $\text{Oracle}(T, R)$ *is a decidable oracle for the correctness question "does* C *implement* T *correctly?" on the finite test corpus generated by the harness: it is total, terminating, and labelling (pass or fail with a structured error from the failing stage).*

Proposition 7.0.17 (prop:topology)**.** $\mathcal{T} = (\text{stages}, \text{policy})$ *is produced only by* derive *(a sealed constructor: no literal construction compiles), and* $\text{topology\_hash} = \mathsf{ca}(\text{stages} \parallel \text{policy} \parallel \text{plan\_hash} \parallel \text{problem\_hash})$ *joins the receipt lineage. Determinism is test-pinned: same plan, same hash.*

Proposition 7.0.18 (prop:totalaccounting)**.** *Every node of every supervised run carries exactly one disposition (*Completed(r)*,* Parked*,* SkippedBy*,* GaveUp*); no silent rows exist (test-pinned:* $|\text{dispositions}| = |V|$*).*

Refusal 7.0.19 (ref:curve)**.** *No novelty-curve-under-faults measurement – yet: member-level fault injection recovers without re-running solver work, so a work-proxy re-measurement would overstate the cache dividend – the flattering error. The claim is withheld until node-level injection is wired through the fleet path; the receipt file names this in its own deferred list.*

Refusal 7.0.20 (refusal:noknhkpath)**.** *No path dependencies on knhk: verified by live probes, recorded as the standing reason – the leanest candidate crate drags tokio/reqwest/opentelemetry transitively; the mu-kernel ships property-testing libraries as regular dependencies and uses unsafe transmutes; the workspace does not build as a whole. Ports of 50–200-line designs, tagged PORT(knhk) with per-item deltas, cost less than the supply chain. Drift is greppable.*

Refusal 7.0.21 (refusal:nomeasuredticks)**.** *No measured CPU ticks: knhk's own hot receipts carried dummy tick values behind a hardcoded 4 GHz assumption. Declared deterministic costs (Ticks as an abstract unit) are the honest model for a planner; rdtsc instrumentation is a bench follow-up, documented as a divergence, not smuggled in as a claim.*

Refusal 7.0.22 (refusal:oneforall)**.** *No* OneForAll*: OTP justifies one-for-all by shared mutable fate between siblings. In an acyclic data-flow plan, same-depth siblings are independent by construction – the coupling one-for-all encodes is inexpressible. The variant does not appear in the Strategy enum at all; an exhaustive-match test makes its future addition a compilation event, forcing the doctrine conversation.*

Refusal 7.0.23 (refusal:simpleoneforone)**.** *No* SimpleOneForOne*; intensity* > 8 *refused, not clamped: derived plans have static step sets, and* `RestartPolicy::new(9, _)` *returns a Refusal – the byte governor applies to recovery itself, and silently clamping a stated policy would be the executor editing the operator's law.*

Refusal 7.0.24 (refusal:v1scope)**.** *v1 scope: beat scheduling, full R1/W1/C1 SLO matrix, quarantine parole, supra-cell supervision. Each receipted with salvage: the enum and one tier transition ship; epoch boundaries already provide the re-admission cadence beats would refine; quarantined classes await a parole mechanism; cells supervising cells is the composition step after this one.*

Remark 7.0.25 (remark:flatthroughput). What flat throughput means: the cost of a supervised fleet tracked its failure novelty, not its failure rate: recovered transients are absorbed at boundary cost, and the one genuinely novel pathology (the crash-looping class) was converted — once, by quorum — into a cheap standing refusal. The full novelty-curve-under-faults re-measurement is receipted as deferred, not claimed.

Remark 7.0.26 (remark:mockmembranes). Mock membranes as differential oracles: the mock membrane produces responses from test fixtures; a real-model run produces a different response. Running both and comparing (differential oracle) bounds the false-accept rate: an implementation error that passes the oracle with the mock response but fails with the real response is caught by the differential.

Theorem 7.0.27 (thm:gap)**.** *`GeometryGap` is the value classification returns when no stored branch matches; it is never a member of any stored list. Consequently no branch ordering, addition, or deletion can capture, mask, or remove the gap outcome: the map admits its own incompleteness by construction, not by convention.*

Proof of 7.0.16. Each stage of the oracle (cargo build, cargo test, cargo clippy, safety audit) terminates on every finite C; the codomain of each stage is $\{0,1\} \times \mathrm{ErrInfo}$, a total finite type; a failure

at any stage short-circuits with the stage index and its ErrInfo, analogous to the staged validator of Paper 04's Definition 4.7. *(by prop:fable-oracle)*
Hence Oracle is total and terminating, and by Proposition 4.4 of Paper 04 any observed failure is a genuine defect in C, not a property of the oracle. *(by prop:fable-oracle)*
□

Proof of 7.0.27. An unstorable value cannot be shadowed by stored values, and first-match-wins terminates in the implicit case exactly when all stored matches fail. *(by def:branch)*
The enforcement is the derivation function's type: branch lists are built from a constructor set that excludes the gap class. *(by def:branch)*
□

Appendix A

Symbol glossary

- B — 04_projection_and_scale
- D — projection_thesis, 00_foundations
- G — synthesis_thesis
- P_ℓ — 04_projection_and_scale
- T — 04_projection_and_scale, 03_planning_geometry
- Γ — synthesis_thesis
- Φ — 02_receipt_cryptography
- Φ_o — 01_admission_algebra
- $\|$ — 02_receipt_cryptography
- α — projection_thesis, 00_foundations
- β — 02_receipt_cryptography
- $\boldsymbol{c}$ — 03_planning_geometry
- $\boldsymbol{m}$ — 03_planning_geometry
- $\boldsymbol{x}$ — 03_planning_geometry
- $\boldsymbol{y}$ — 03_planning_geometry
- $\perp$ — projection_thesis, 00_foundations
- κ — projection_thesis, 04_projection_and_scale, 03_planning_geometry, 01_admission_algebra
- $\mathbb{D}$ — 01_admission_algebra
- **Life** — 00_foundations
- **N** — 03_planning_geometry
- **b** — 04_projection_and_scale

- $\mathcal{A}$ — projection_thesis, 00_foundations
- $\mathcal{C}$ — 01_admission_algebra
- $\mathcal{L}\lceil\}$ — 02_receipt_cryptography
- $\mathcal{L}$ — 02_receipt_cryptography
- $\mathcal{O}$ — projection_thesis, 00_foundations
- $\mathcal{O}^*$ — projection_thesis, 00_foundations
- $\mathcal{P}$ — 03_planning_geometry
- $\mathcal{R}$ — 00_foundations
- $\mathcal{R}_p$ — projection_thesis
- $\mathcal{S}$ — 01_admission_algebra
- $\mathcal{T}$ — synthesis_thesis
- Kill — 04_projection_and_scale
- MRR — 01_admission_algebra
- SymId — 03_planning_geometry
- cat — 01_admission_algebra
- dg — synthesis_thesis, projection_thesis, 02_receipt_cryptography
- lane — 01_admission_algebra
- scn — 01_admission_algebra
- ADMITTED — 01_admission_algebra
- H — synthesis_thesis, projection_thesis, 00_foundations
- Over — 04_projection_and_scale
- ca — synthesis_thesis
- fr — synthesis_thesis, 02_receipt_cryptography
- g — 02_receipt_cryptography
- s — 04_projection_and_scale
- μ — projection_thesis, 00_foundations
- π — projection_thesis
- π_f — 01_admission_algebra
- BRCE — 00_foundations

- φ — projection_thesis, 03_planning_geometry, 02_receipt_cryptography, 00_foundations
- k — 02_receipt_cryptography
- t_{H} — 04_projection_and_scale